

수중 UUV VIO-SLAM을 위한 VINS-Fusion 코드 분석

under_water_image_match 프로젝트 정리

May 8, 2026

1 왜 VINS-Fusion을 분석하는가

수중 UUV가 수영장이나 저시야 환경에서 자기 위치와 자세를 추정하려면 단순 image matching 만으로는 부족하다. 반복 타일, 낮은 텍스처, blur, 조명 변화 때문에 feature가 끊기고, stereo depth 는 calibration과 disparity 오차에 민감하며, 장시간 주행하면 local odometry가 drift 한다. 따라서 실전 구조는 다음 네 가지가 함께 있어야 한다.

1. 영상에서 feature를 안정적으로 추적한다.
2. stereo 또는 motion으로 feature depth를 만든다.
3. IMU로 feature 부족 구간의 motion prior를 제공한다.
4. loop, sonar, depth, map 같은 global constraint로 drift를 줄인다.

VINS-Fusion은 이 흐름을 실제 C++ 코드로 구현한 대표적인 VIO/SLAM 코드베이스다. 이 문서는 GUI 사용법이 아니라, **VINS-Fusion** 원본 코드가 어떤 수식과 파라미터로 pose를 추정하는지를 분석한다. GUI와 MP4 변환 코드는 마지막에 wrapper로만 다룬다.

2 실제 코드 범위

분석 대상은 현재 workspace의 third_party/VINS-Fusion 이다. 해당 디렉터리는 <https://github.com/HKUST-Aerial-Robotics/VINS-Fusion.git>를 remote로 가지며, 현재 checkout commit은 be55a937a57436548ddfb1bd324bc1e9a9e828e0 이다.

모듈	역할
vins_estimator	VIO의 중심. image/IMU 입력, feature tracking, initialization, sliding-window optimization, marginalization을 수행한다.
featureTracker	KLT optical flow 기반 feature frontend.
factor	visual projection factor, IMU factor, marginalization factor 등 Ceres residual 구현.
loop_fusion	loop closure와 pose graph optimization. local VIO를 SLAM에 가깝게 확장하는 부분.
camera_models	camera projection, undistortion 모델. 수중 calibration과 직접 연결된다.

3 처음 보는 사람을 위한 핵심 용어

핵심 정의: VO, VIO, SLAM의 차이

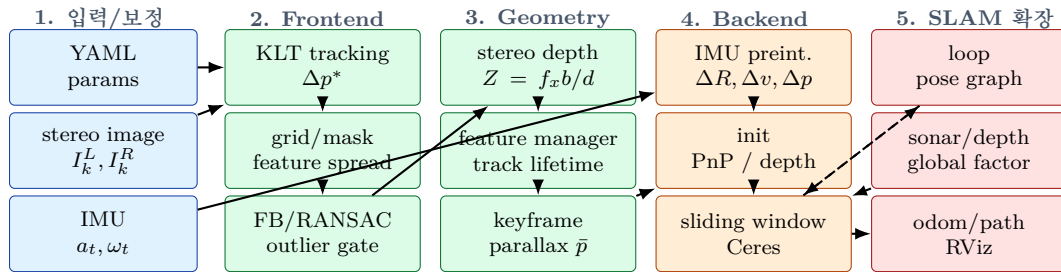
VO(Visual Odometry)는 camera image만으로 이전 위치에서 현재 위치로 얼마나 움직였는지 추정한다. **VIO(Visual-Inertial Odometry)**는 image에 IMU를 더해 feature가 부족하거나 blur가 있어도 motion prior를 얻는다. **SLAM**은 local odometry에 loop, sonar, marker, map 같은 global constraint를 더해 장시간 drift를 줄인다.

간단 예시: 수영장 UUV 상황

UUV가 타일 벽을 따라 10 m 이동한다고 하자. VO는 image feature만 보고 움직임을 추정하므로 반복 타일에서 잘못 붙을 수 있다. VIO는 IMU가 “지금 회전/가속이 이 정도였다”는 힌트를 준다. SLAM은 나중에 같은 부표나 marker를 다시 봤을 때 “아까 본 곳으로 돌아왔다”는 제약을 추가해 누적 drift를 줄인다.

용어	짧은 정의	이 문서에서 보는 위치
feature	이미지에서 추적할 점. corner, texture point 등.	<code>feature_tracker.cpp</code> 의 KLT tracking.
track	같은 feature가 여러 frame에서 이어진 기록.	feature lifetime, parallax, keyframe 판단.
correspondence	서로 다른 두 이미지에서 같은 3D 점이라고 가정한 2D 점 쌍.	KLT, stereo matching, RANSAC.
inlier/outlier	geometry model을 만족하는 점/만족하지 않는 점.	RANSAC gate, robust loss.
residual	예측과 실제 관측의 차이. 작을수록 모델이 관측을 잘 설명한다.	projection factor, IMU factor, prior factor.
factor graph	상태 변수와 residual 제약을 그래프로 표현한 것.	sliding-window optimization.
inverse depth	depth Z 대신 $\lambda = 1/Z$ 를 쓰는 표현.	feature depth 최적화.
marginalization	window 밖으로 나가는 변수를 prior로 압축하는 과정.	sliding window 유지.

4 전체 파이프라인



이 문서는 위 흐름을 그대로 따른다. 각 단계는 목적 → 색상 수식 → 파라미터 영향 → 실제 코드 위치 → 수정 튜닝 포인트 순서로 읽는다.

그림 1. 문서 전체 구조를 VINS 실행 파이프라인 순서로 재배치한 그림. 앞쪽은 feature와 depth를 만드는 frontend, 뒤쪽은 residual을 묶어 푸는 backend, 마지막은 SLAM/global correction이다.

단계	핵심 코드	핵심 질문
Config	parameters.cpp	어떤 파라미터가 어떤 수식항에 연결되는가?
Feature tracking	feature_tracker.cpp	수중 반복 패턴에서 feature가 안정적으로 살아남는가?
Stereo depth	feature_manager.cpp	baseline/intrinsic/disparity가 depth scale을 얼마나 흔드는가?
IMU	integration_base.h, imu_factor.h	camera와 IMU residual이 같은 motion을 설명하는가?
Optimization	estimator.cpp, projection*.cpp	visual/IMU/prior residual을 동시에 최소화하는가?
SLAM extension	loop_fusion, global factor	sonar/ local drift를 외부 constraint로 잡을 수 있는가?

■ 상태/최적화 변수 ■ 센서 관측 ■ 파라미터/노이즈 ■ residual/error

현재 생성 config 기준	가	읽는 기준
max_cnt	150	반복 타일이 많으면 150을 그대로 올리기보다 min_dist, flow_back, RANSAC gate를 먼저 본다.
min_dist	30 px	같은 타일 무늬에 feature가 물리는 것을 막는 시작값. 848x480 영상에서는 무난한 보수값이다.
F_threshold	1.0 px	RANSAC을 켜올 때 pixel 단위 epipolar gate. 현재 VINS 코드 경로에서는 호출이 꺼져 있으므로 별도 활성화가 필요하다.
keyframe_parallax	10 px	VINS 기본 stereo/mono-IMU config와 같은 시작값. 수영장치처럼 texture가 약하면 실제 track lifetime과 같이 조정한다.
max_solver_time	0.04 s	realtime을 위한 backend budget. Docker/Mac 실험에서는 결과 품질보다 실행 가능성을 먼저 보는 값이다.

5 0단계: Config가 estimator의 전역 파라미터가 된다

핵심 개념

핵심 정의: Config

Config는 알고리즘의 “실험 조건표”다. feature를 몇 개까지 뽑을지, camera calibration을 무엇으로 볼지, solver를 몇 초까지 돌릴지 같은 값이 여기서 정해진다. 코드에서는 이 값들이 전역 변수로 읽힌 뒤 frontend와 backend에서 반복 사용된다.

간단 예시: 파라미터 하나가 결과를 바꾸는 방식

max_cnt=80이면 backend가 사용할 visual residual 후보가 적어진다. max_cnt=200이면 후보는 늘지만, 수영장 타일처럼 반복 패턴이 많을 때 outlier도 함께 늘 수 있다. 따라서 “feature 수가 많다”는 말은 “좋은 pose constraint가 많다”와 같지 않다.

VINS-Fusion의 많은 튜닝 값은 YAML에서 읽혀 전역 변수로 저장된다. 따라서 GUI에서 값을 바꾸든 YAML을 직접 수정하든, 최종적으로는 readParameters()가 읽는 값으로 들어간다.

YAML config $\rightarrow$ {MAX_CNT, MIN_DIST, F_THRESHOLD, MIN_PARALLAX, SOLVER_TIME, ...} $\rightarrow$ frontend/backend

파라미터 연결 지도

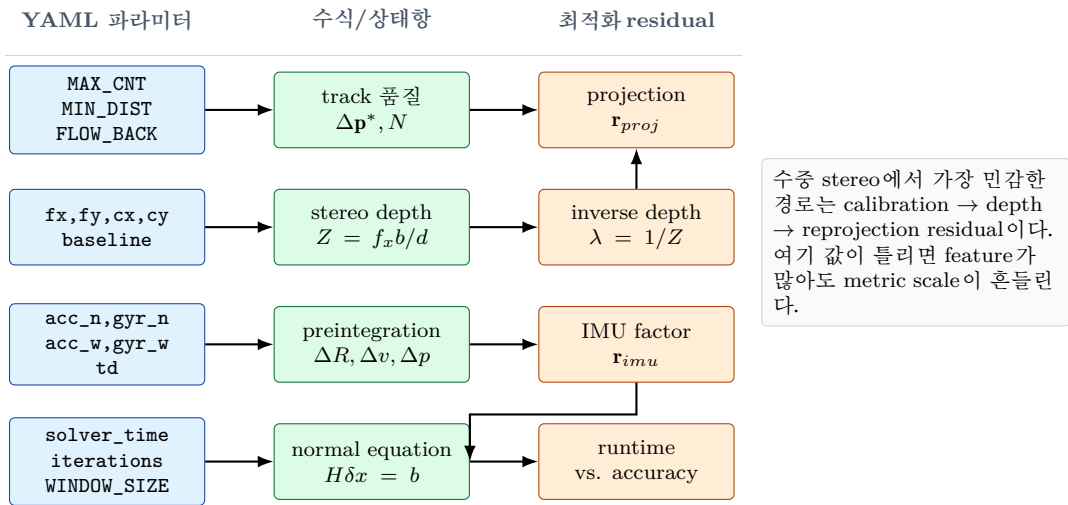


그림 2. 튜닝 파라미터가 단순 옵션이 아니라 residual의 weight, 상태 변수, geometry scale에 직접 연결되는 구조.

핵심 파라미터

파라미터	직접 연결되는 항	영향
MAX_CNT	feature 수 N	visual residual 개수를 바꾼다. 너무 작으면 constraint 부족, 너무 크면 outlier와 계산량 증가.
MIN_DIST	feature 공간 분포	feature가 한 타일/벽면에 몰리는 것을 막는다. 너무 크면 feature 부족.
FLOW_BACK	왕복 추적 오차	KLT outlier를 제거한다. 수중 반복 패턴에서는 켜는 것이 안전하다.
F_THRESHOLD	epipolar RANSAC threshold	현재 코드에서는 <code>rejectWithF()</code> 가 주석 처리되어 직접 적용되지 않는다. 켜면 threshold가 작을수록 엄격하다.
keyframe_parallax	$\bar{p}$, MIN_PARALLAX	keyframe 빈도와 marginalization 방향을 바꾼다.
max_solver_time	Ceres time budget	realtime성과 수렴 품질 사이 trade-off.
max_num_iterations	Ceres iteration count	수렴 전 종료 여부와 계산량에 영향.

실제 코드

읽는 포인트: YAML key가 전역 변수로 들어가고, 이후 frontend/backend 코드가 같은 값을 공유한다.

parameters.cpp:66–108

```

66 void readParameters(std::string config_file)
67 {
68     FILE *fh = fopen(config_file.c_str(), "r");
69     if(fh == NULL){
70         ROS_WARN("config_file doesn't exist; wrong config_file path");
71         ROS_BREAK();
72         return;
73     }
74     fclose(fh);
75
76     cv::FileStorage fsSettings(config_file, cv::FileStorage::READ);
77     if(!fsSettings.isOpened())
78     {
79         std::cerr << "ERROR: Wrong path to settings" << std::endl;
80     }
81
82     fsSettings["image0_topic"] >> IMAGE0_TOPIC;
83     fsSettings["image1_topic"] >> IMAGE1_TOPIC;
84     MAX_CNT = fsSettings["max_cnt"];
85     MIN_DIST = fsSettings["min_dist"];
86     F_THRESHOLD = fsSettings["F_threshold"];
87     SHOW_TRACK = fsSettings["show_track"];
88     FLOW_BACK = fsSettings["flow_back"];
89
90     MULTIPLE_THREAD = fsSettings["multiple_thread"];
91
92     USE_IMU = fsSettings["imu"];
93     printf("USE_IMU: %d\n", USE_IMU);
94     if(USE_IMU)
95     {
96         fsSettings["imu_topic"] >> IMU_TOPIC;
97         printf("IMU_TOPIC: %s\n", IMU_TOPIC.c_str());
98         ACC_N = fsSettings["acc_n"];
99         ACC_W = fsSettings["acc_w"];
100         GYR_N = fsSettings["gyr_n"];
101         GYR_W = fsSettings["gyr_w"];
102         G.z() = fsSettings["g_norm"];
103     }
104
105     SOLVER_TIME = fsSettings["max_solver_time"];
106     NUM_ITERATIONS = fsSettings["max_num_iterations"];
107     MIN_PARALLAX = fsSettings["keyframe_parallax"];
108     MIN_PARALLAX = MIN_PARALLAX / FOCAL_LENGTH;

```

코드 분석

- MAX_CNT, MIN_DIST, F_THRESHOLD는 feature frontend의 입력 품질을 정한다. 즉 backend 수식보다 앞에서 residual 후보 자체를 바꾸는 값이다.
- MIN_PARALLAX는 YAML의 pixel 값 keyframe_parallax를 FOCAL_LENGTH로 나눈 normalized threshold다. 그래서 image resize나 focal 설정이 바뀌면 같은 10 px도 다른 geometry 의미가 된다.
- SOLVER_TIME, NUM_ITERATIONS는 Ceres가 얼마나 오래 $H\delta x = b$ 를 풀지 정한다. feature가 나쁘면 이 값을 키워도 outlier를 정확하게 바꾸지는 못한다.
- 수중 UUV에서는 이 함수가 읽는 값이 실험 조건표의 기준이다. GUI 값, YAML 값, 실제 estimator 전역 변수가 서로 다른지 먼저 확인해야 한다.

6 1단계: Sensor input과 timestamp

핵심 정의: Sensor input과 timestamp

Sensor input은 camera image와 IMU measurement 자체이고, timestamp는 그 측정이 발생한 시간이다. VIO에서는 값의 크기만큼이나 “언제 측정됐는가”가 중요하다. camera와 IMU 시간이 어긋나면 같은 motion을 서로 다른 시간에서 설명하려고 해서 residual이 커진다.

간단 예시: 시간 오차가 만드는 문제

UUV가 빠르게 yaw 회전하는 순간을 생각하자. camera frame은 $t = 1.00$ 초인데 IMU가 $t = 0.95$ 초 motion으로 연결되면, visual은 이미 회전한 장면을 보고 IMU는 회전 전 motion을 주장한다. 이때 backend는 둘을 동시에 만족시키려다 pose가 흔들릴 수 있다.

수식

VIO에서 입력은 camera frame과 IMU measurement다.

$$I_k \text{ at } t_k, \quad \mathbf{u}_t = \begin{bmatrix} \mathbf{a}_t \\ \boldsymbol{\omega}_t \end{bmatrix}$$

각 항의 의미

항	의미
I_k	k 번째 camera frame. stereo라면 left/right pair가 같은 timestamp에 있어야 한다.
t_k	camera timestamp. IMU preintegration 구간을 자르는 기준이다.
$\mathbf{a}_t$	accelerometer 측정값. 실제 가속도, 중력, bias, noise가 섞인다.
$\boldsymbol{\omega}_t$	gyroscope 측정값. 각속도, bias, noise가 섞인다.

파라미터와 영향

파라미터	수식/코드 항	작계/느리면	크게/빠르면
camera FPS	$t_k - t_{k-1}$	frame 간 motion 이 커져 KLT 실패 가능성 증가.	계산량 증가, tracking은 쉬워짐.
IMU rate	Δt	빠른 회전/가속을 놓쳐 preintegration 이 거칠어 짐.	motion prior는 좋아지나 처리량 증가.
time offset td	image time 보정	값의 대소보다 정확성이 중요. 틀리면 visual residual과 IMU residual 이 충돌.	동일. 빠른 motion 일수록 민감.

실제 코드

KITTIOdomTest.cpp:88-113

```

88     ss << setfill('0') << setw(6) << i;
89     leftImagePath = dataPath + "image_0/" + ss.str() + ".png";
90     rightImagePath = dataPath + "image_1/" + ss.str() + ".png";
91     //printf("%lu %f \n", i, imageTimeList[i]);
92     //printf("%s\n", leftImagePath.c_str() );
93     //printf("%s\n", rightImagePath.c_str() );
94
95     imLeft = cv::imread(leftImagePath, CV_LOAD_IMAGE_GRAYSCALE );
96     sensor_msgs::ImagePtr imLeftMsg = cv_bridge::CvImage(std_msgs::Header(), "mono8", imLeft).
toImageMsg();
97     imLeftMsg->header.stamp = ros::Time(imageTimeList[i]);
98     pubLeftImage.publish(imLeftMsg);
99
100    imRight = cv::imread(rightImagePath, CV_LOAD_IMAGE_GRAYSCALE );
101    sensor_msgs::ImagePtr imRightMsg = cv_bridge::CvImage(std_msgs::Header(), "mono8", imRight).
toImageMsg();
102    imRightMsg->header.stamp = ros::Time(imageTimeList[i]);
103    pubRightImage.publish(imRightMsg);
104
105
106    estimator.inputImage(imageTimeList[i], imLeft, imRight);
107
108    Eigen::Matrix<double, 4, 4> pose;
109    estimator.getPoseInWorldFrame(pose);
110    if(outFile != NULL)
111        fprintf (outFile, "%f %f %f %f %f %f %f %f %f %f %f %f \n",pose(0,0), pose(0,1), pose(0,2),pose
(0,3),
112                                     pose(1,0), pose(1,1), pose(1,2),pose(1,3),
113                                     pose(2,0), pose(2,1), pose(2,2),pose(2,3));

```

코드 분석

- 이 코드는 KITTI-style stereo image sequence를 시간 순서대로 읽고 estimator에 넣는 테스트 runner다. 즉 ROS topic이 아니라 파일 기반 실험 입구다.
- left/right image가 같은 index와 timestamp로 들어간다는 가정이 있다. MP4에서 frame drop이나 길이 차이가 있으면 여기서부터 stereo correspondence가 틀어진다.
- 수중 실험에서는 이 runner가 실제 센서 동기화를 보장하지 않는다. export 단계에서 left/right frame count, FPS, timestamp 간격을 검증해야 한다.

estimator.cpp:160-194

```

160 void Estimator::inputImage(double t, const cv::Mat &img, const cv::Mat &img1)
161 {
162     inputImageCnt++;
163     map<int, vector<pair<int, Eigen::Matrix<double, 7, 1>>> featureFrame;

```



```

164     TicToc featureTrackerTime;
165
166     if(_img1.empty())
167         featureFrame = featureTracker.trackImage(t, _img);
168     else
169         featureFrame = featureTracker.trackImage(t, _img, _img1);
170     //printf("featureTracker time: %f\n", featureTrackerTime.toc());
171
172     if (SHOW_TRACK)
173     {
174         cv::Mat imgTrack = featureTracker.getTrackImage();
175         pubTrackImage(imgTrack, t);
176     }
177
178     if(MULTIPLE_THREAD)
179     {
180         if(inputImageCnt % 2 == 0)
181         {
182             mBuf.lock();
183             featureBuf.push(make_pair(t, featureFrame));
184             mBuf.unlock();
185         }
186     }
187     else
188     {
189         mBuf.lock();
190         featureBuf.push(make_pair(t, featureFrame));
191         mBuf.unlock();
192         TicToc processTime;
193         processMeasurements();
194         printf("process time: %f\n", processTime.toc());

```

코드 분석

- image callback에서 feature tracker 결과가 estimator의 measurement buffer로 들어간다. 이 시점부터 raw image가 아니라 feature observation map이 backend 입력이다.
- 여기서 품질이 낮은 feature가 들어오면 뒤의 PnP, triangulation, Ceres residual이 모두 오염된다. backend에서 해결할 문제가 아니라 frontend gate 문제로 봐야 한다.
- timestamp는 IMU preintegration 구간을 자르는 기준이다. td나 frame time이 틀리면 visual residual과 IMU residual이 서로 다른 motion을 주장한다.

estimator.cpp:199-205

```

199 void Estimator::inputIMU(double t, const Vector3d &linearAcceleration, const Vector3d &angularVelocity)
200 {
201     mBuf.lock();
202     accBuf.push(make_pair(t, linearAcceleration));
203     gyrBuf.push(make_pair(t, angularVelocity));
204     //printf("input imu with time %f \n", t);
205     mBuf.unlock();

```

코드 분석

- IMU measurement는 image와 별도 buffer로 들어가고, 이후 image timestamp 사이의 IMU sample들이 preintegration에 쓰인다.
- 현재 MP4-only 실험에서는 imu: 0이면 이 경로가 실제 pose 추정에 기여하지 않는다. 그래서 지금 결과는 엄밀히는 VIO가 아니라 stereo visual odometry에 가깝다.
- 실제 UUV로 넘어가려면 IMU rate, time offset, camera-IMU extrinsic을 이 입력 경로 기준으로 맞춰야 한다.

7 2단계: Feature tracking frontend

핵심 정의: Feature tracking

Feature tracking은 한 이미지에서 잡은 점이 다음 이미지의 어디로 이동했는지 찾는 단계다. VINS-Fusion의 기본 frontend는 descriptor matching보다 KLT optical flow를 중심으로 한다. 즉 feature 주변 작은 patch의 밝기 패턴이 다음 frame에서도 비슷하다고 보고 이동량 Δp 를 찾는다.

간단 예시: 수영장 타일에서 왜 어려운가

타일 모서리 하나를 feature로 잡았다고 하자. 다음 frame에 비슷한 타일 모서리가 여러 개 있으면 KLT는 진짜 같은 점이 아니라 옆 타일의 비슷한 점에 붙을 수 있다. 그래서 forward-backward check, RANSAC, grid 분산, track lifetime 검사가 필요하다.

수식

VINS-Fusion frontend는 descriptor matching이 아니라 KLT optical flow 중심이다. feature 주변 patch의 밝기 패턴이 다음 frame에서도 유지된다고 보고 displacement를 찾는다.

$$\Delta \mathbf{p}^* = \arg \min_{\Delta \mathbf{p}} \sum_{\mathbf{q} \in \Omega} [I_{k+1}(\mathbf{q} + \mathbf{p} + \Delta \mathbf{p}) - I_k(\mathbf{q} + \mathbf{p})]^2$$

선형화하면 다음 normal equation으로 볼 수 있다.

$$G \Delta \mathbf{p} = \mathbf{b}, \quad G = \sum_{\mathbf{q} \in \Omega} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

각 항의 의미

항	의미
$\mathbf{p}$	이전 frame의 feature 위치.
$\Delta \mathbf{p}^*$	KLT가 찾는 최적 2D 이동량.
Ω	feature 주변 patch window. 코드에서는 <code>cv::Size(21, 21)</code> .
I_k, I_{k+1}	이전/현재 grayscale intensity.
G	patch gradient matrix. 두 eigenvalue가 모두 충분히 커야 corner로 안정적이다.
$\lambda_{\min}(G)$	Shi-Tomasi corner score에 해당하는 값. 작으면 low-texture/edge feature가 된다.

시각적 해석

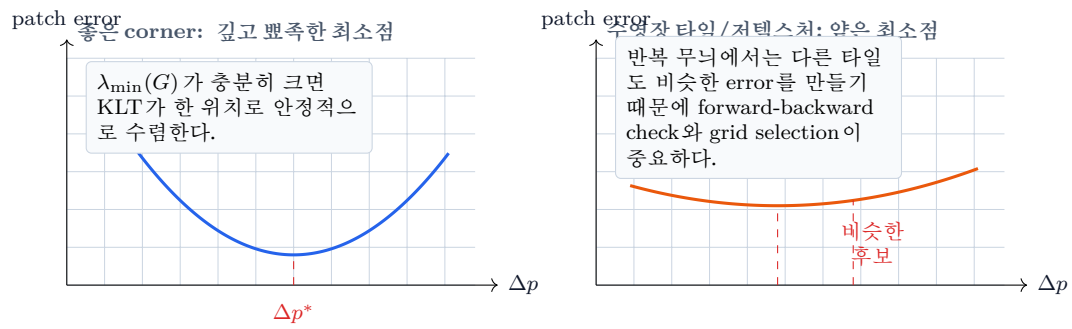


그림 3. KLT는 patch error가 가장 작아지는 이동량을 찾는다. 저텍스처/반복 패턴에서는 error basin이 얇아서 outlier가 쉽게 통과한다.

파라미터와 영향

파라미터	수식/항과의 관계	작게 하면	크게 하면
MAX_CNT	feature residual 후보 수 N	constraint 부족, tracking lost 위험.	정보는 많아지지만 outlier와 계산량 증가. 반복 타일에서 위험.
MIN_DIST	어떤 patch Ω 를 feature로 선택 할지 결정	feature가 촘촘해지고 한 패턴에 몰림.	feature가 넓게 퍼지지만 저텍스처에서는 개수 부족.
FLOW_BACK	$\ \mathbf{p}_{k-1} - \hat{\mathbf{p}}_{k-1}\ $ 검증	끄면 빠르지만 오추적 통과.	켜면 느리지만 outlier 감소.
LK window	$G = \sum_{\Omega} \nabla I \nabla I^T$ 의 영역	noise/blur에 약함.	큰 motion에는 강하지만 다른 타일 패턴이 섞일 수 있음.

수중 영상에서 보는 진단 숫자

지표	위험 신호	조치
tracked feature 수	30-50개 이하로 자주 하락	MAX_CNT만 올리지 말고 MIN_DIST를 낮추거나 조명/blur/ROI를 먼저 확인한다.
new feature 비율	매 frame 새 feature가 대부분	기존 track lifetime이 짧다는 뜻이다. FLOW_BACK 결과와 motion blur를 확인한다.
같은 타일 영역 집중	feature가 한 벽면/한 줄 타일에 몰림	MIN_DIST 증가, grid-based mask 강화, low-texture/repeated-pattern rejection 추가.
RANSAC inlier ratio	0.5 이하로 반복	threshold를 키우기 전에 correspondence가 반복 타일에 잘못 붙는지 먼저 본다.

RANSAC gate 시각화

핵심 정의: RANSAC gate

RANSAC gate는 많은 correspondence 중에서 하나의 geometry model을 잘 만족하는 점만 inlier로 남기는 필터다. 여기서는 fundamental matrix F 가 만드는 epipolar line을 기준으로, 점과 선 사이 거리 d_{epi} 가 threshold보다 작은 correspondence만 통과시킨다.

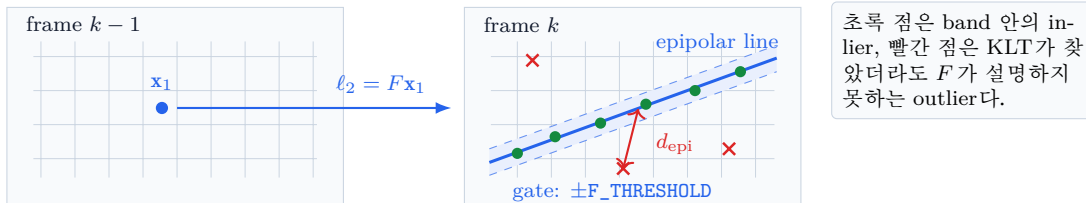
간단 예시: 타일 오매칭 제거

왼쪽 이미지의 한 타일 모서리가 오른쪽 이미지에서 비슷한 다른 타일 모서리에 잘못 붙었다고 하자. KLT만 보면 patch가 비슷해서 성공처럼 보일 수 있다. 하지만 epipolar line에서 멀리 떨어져 있으면 d_{epi} 가 커지고, RANSAC gate에서 outlier로 제거된다.

VINS-Fusion 원본의 `rejectWithF()`는 epipolar geometry로 outlier를 제거하는 함수지만, 현재 stereo 흐름에서는 호출이 주석 처리되어 있다. 켜서 사용한다면 핵심 gate는 epipolar line까지의 거리다.

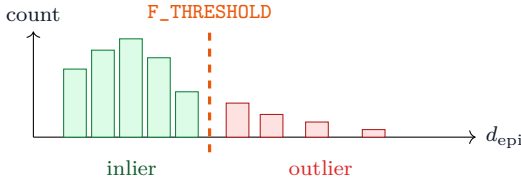
$$d_{\text{epi}} = \frac{|\mathbf{x}_2^\top F \mathbf{x}_1|}{\sqrt{(F \mathbf{x}_1)_1^2 + (F \mathbf{x}_1)_2^2}}, \quad d_{\text{epi}} < F_THRESHOLD \Rightarrow \text{inlier}$$

A. 한 feature가 만드는 epipolar gate



반복 타일에서는 KLT score가 좋아도 d_{epi} 가 커질 수 있으므로, 이 gate가 backend로 들어갈 correspondence를 한 번 더 거른다.

B. residual을 threshold로 자르는 방식



C. threshold 대신에 따른 결과

너무 작음: 정상점 reject → track 부족

적절함: geometry inlier만 backend 전달

너무 큼: 반복 타일 outlier 통과 → pose jump

따라서 inlier 개수가 많다고 항상 좋은 것이 아니다. $F_THRESHOLD$ 는 feature를 많이 살리는 파라미터가 아니라, $\mathbf{x}_2^\top F \mathbf{x}_1 = 0$ 모델을 얼마나 엄격하게 적용할지 정하는 gate다.

그림 4. RANSAC gate의 실제 의미. F 가 만든 epipolar line 주변의 band 안에 들어오는 correspondence만 inlier가 된다. threshold는 residual histogram을 자르는 기준이며, 작으면 정상점 손실, 크면 outlier 통과가 생긴다.

실제 코드

읽는 포인트: 여기서 실제 KLT 추적, track status filtering, forward-backward check가 한 번에 보인다.

feature_tracker.cpp:120-149

```
120         cv::calcOpticalFlowPyrLK(prev_img, cur_img, prev_pts, cur_pts, status, err, cv::Size(21, 21),
121             1,
122             cv::TermCriteria(cv::TermCriteria::COUNT+cv::TermCriteria::EPS, 30, 0.01), cv::
123             OPTFLOW_USE_INITIAL_FLOW);
124
125         int succ_num = 0;
126         for (size_t i = 0; i < status.size(); i++)
127         {
128             if (status[i])
129                 succ_num++;
130         }
131         if (succ_num < 10)
132             cv::calcOpticalFlowPyrLK(prev_img, cur_img, prev_pts, cur_pts, status, err, cv::Size(21,
133             21), 3);
134         else
135             cv::calcOpticalFlowPyrLK(prev_img, cur_img, prev_pts, cur_pts, status, err, cv::Size(21, 21),
136             3);
137         // reverse check
138         if (FLOW_BACK)
139         {
140             vector<uchar> reverse_status;
141             vector<cv::Point2f> reverse_pts = prev_pts;
142             cv::calcOpticalFlowPyrLK(cur_img, prev_img, cur_pts, reverse_pts, reverse_status, err, cv::
143             Size(21, 21), 1,
144             cv::TermCriteria(cv::TermCriteria::COUNT+cv::TermCriteria::EPS, 30, 0.01), cv::
145             OPTFLOW_USE_INITIAL_FLOW);
146             //cv::calcOpticalFlowPyrLK(cur_img, prev_img, cur_pts, reverse_pts, reverse_status, err, cv::
147             Size(21, 21), 3);
148             for (size_t i = 0; i < status.size(); i++)
149             {
150                 if (status[i] && reverse_status[i] && distance(prev_pts[i], reverse_pts[i]) <= 0.5)
151                 {
152                     status[i] = 1;
153                 }
154                 else
155                     status[i] = 0;
156             }
157         }
```

코드 분석

- 핵심은 calcOpticalFlowPyrLK다. 이전 frame의 cur_pts를 다음 frame으로 이동시키며 Δp^* 를 수치적으로 찾는다.
- status filtering은 KLT가 실패한 점을 제거한다. 이 단계에서 살아남은 점만 이후 visual residual 후보가 된다.
- FLOW_BACK이 켜지면 현재 frame에서 다시 이전 frame으로 역추적한다. 왕복 위치가 다르면 반복 타일에 잘못 붙은 점일 가능성이 커서 제거한다.
- 수영장 타일 환경에서는 이 블록이 가장 중요하다. feature 개수를 늘리는 것보다 왕복 오차와 track lifetime을 먼저 봐야 한다.

feature_tracker.cpp:167-184

```
167     if (1)
168     {
169         //rejectWithF();
170         ROS_DEBUG("set mask begins");
171         TicToc t_m;
172         setMask();
173         ROS_DEBUG("set mask costs %fms", t_m.toc());
174
175         ROS_DEBUG("detect feature begins");
176         TicToc t_t;
177         int n_max_cnt = MAX_CNT - static_cast<int>(cur_pts.size());
178         if (n_max_cnt > 0)
179         {
180             if (mask.empty())
181                 cout << "mask is empty " << endl;
182             if (mask.type() != CV_8UC1)
183                 cout << "mask type wrong " << endl;
```

코드 분석

- `setMask()` 이후 새 feature를 뽑기 때문에, 이미 존재하는 feature 주변에는 새 점이 과하게 몰리지 않는다. 이것이 grid-based selection과 비슷한 역할을 한다.
- `MAX_CNT - cur_pts.size()` 만큼 부족한 feature를 새로 채운다. 따라서 `MAX_CNT`는 항상 feature를 많이 쓰겠다는 뜻이 아니라 “부족하면 보충할 상한”이다.
- `MIN_DIST`가 작으면 같은 타일 줄에 feature가 몰릴 수 있고, 크면 저텍스처 구간에서 feature가 부족해진다. 수중에서는 이 값을 단독으로 보지 말고 feature 분포 이미지를 같이 봐야 한다.

feature_tracker.cpp:326-331

```

326         un_prev_pts[i] = cv::Point2f(tmp_p.x(), tmp_p.y());
327     }
328
329     vector<uchar> status;
330     cv::findFundamentalMat(un_cur_pts, un_prev_pts, cv::FM_RANSAC, F_THRESHOLD, 0.99, status);
331     int size_a = cur_pts.size();

```

코드 분석

- 이 함수는 `findFundamentalMat(..., cv::FM_RANSAC, F_THRESHOLD, 0.99)`로 epipolar geometry outlier를 제거한다.
- 중요한 점은 현재 실험 경로에서 이 함수 호출이 꺼져 있다는 것이다. 그래서 YAML의 `F_threshold`를 바꿔도 실제 tracking 결과가 바로 바뀌지 않을 수 있다.
- RANSAC을 켜면 repeated pattern mismatch를 줄일 수 있지만, blur와 굴절 때문에 정상 대응점도 reject될 수 있다. inlier ratio와 RViz path jump를 같이 봐야 한다.

8 3단계: Stereo depth와 triangulation

핵심 정의: Stereo depth

Stereo depth는 왼쪽 camera와 오른쪽 camera에서 같은 feature가 보이는 위치 차이, 즉 disparity d 로 깊이 Z 를 계산하는 방법이다. 두 camera 사이 baseline b 와 focal length f_x 를 알고 있으면 $Z = f_x b / d$ 로 거리를 추정할 수 있다.

간단 예시: disparity 1 pixel 오차의 의미

가까운 점은 disparity가 커서 1 pixel 오차가 depth에 작게 반영된다. 먼 벽면은 disparity가 작기 때문에 같은 1 pixel 오차가 큰 depth 오차로 증폭된다. 수영장 벽처럼 멀고 반복적인 texture는 stereo depth가 가장 흔들리기 쉬운 대상이다.

수식

rectified stereo의 직관적인 depth 식은 다음과 같다.

$$d = u_L - u_R, \quad Z = \frac{f_x b}{d}$$

disparity 오차가 depth 오차로 전파되는 근사는 다음과 같다.

$$\sigma_Z \approx \left| \frac{\partial Z}{\partial d} \right| \sigma_d = \frac{f_x b}{d^2} \sigma_d$$

VINS-Fusion 내부에서는 일반적인 두 projection matrix로 DLT triangulation을 수행한다.

$$AX = 0, \quad X^* = \arg \min_{\|X\|=1} \|AX\|$$

각 항의 의미

항	의미
u_L, u_R	left/right image에서 같은 feature의 x 좌표.
d	disparity. 작을수록 먼 점이고 depth 오차가 커진다.
f_x	x축 focal length. resize와 calibration에 따라 달라진다.
b	stereo baseline. metric scale을 결정한다.
Z	camera 기준 feature depth. backend에서는 inverse depth $\lambda = 1/Z$ 로 자주 사용.
$A, \mathbf{X}$	DLT triangulation 행렬과 homogeneous 3D point.

민감도 그래프

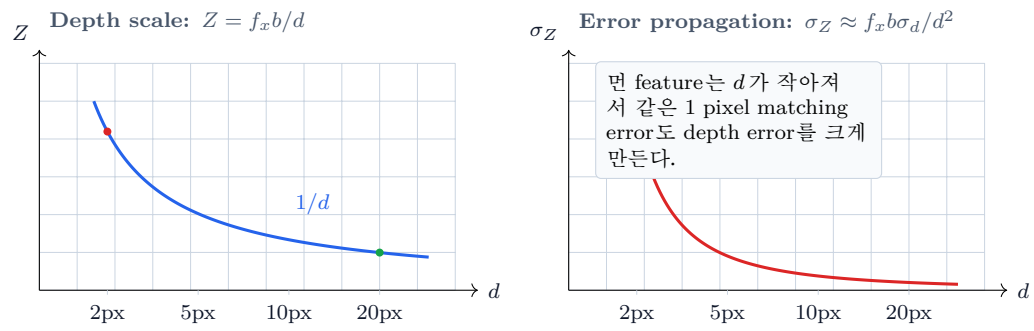


그림 5. stereo는 가까운 점보다 먼 점에서 훨씬 불안정하다. 수중에서 calibration과 right-left matching 품질을 먼저 검증해야 하는 이유다.

파라미터와 영향

파라미터	수식항 관계	작게 하면	크게 하면
baseline b	Z 에 선형으로 곱해짐	depth/trajectory scale이 작아짐.	depth/trajectory scale이 커짐. 실제보다 크게 넣으면 경로가 부풀어 보임.
f_x	Z 에 선형으로 곱해짐	depth가 작아지고 scale 축소.	depth가 커지고 scale 확대.
disparity d	Z 의 분모	작은 d 는 먼 점. 작은 pixel 오차가 큰 depth 오차로 증폭.	가까운 점은 depth가 안정적이지만 occlusion 가능성 증가.
resize_width	pixel 좌표와 intrinsic 단위 변경	빠르지만 disparity 정밀도 감소.	정밀도는 좋아지지만 처리량 증가. intrinsic도 같이 scaling해야 함.

현재 config로 보는 depth 오차 예시

현재 생성된 pinhole config는 $f_x = 525.76$, baseline은 $b = 0.05$ m로 들어가 있다. 이 값이 정확한 수중 calibration이라는 뜻은 아니고, stereo depth 민감도를 보는 기준값이다.

$$Z = \frac{525.76 \cdot 0.05}{d}, \quad \sigma_Z \approx \frac{525.76 \cdot 0.05}{d^2} \sigma_d$$

disparity d	depth Z	$\sigma_d = 0.5$ px일 때	해석
5 px	약 5.26 m	$\sigma_Z \approx 0.53$ m	먼 벽/타일 feature는 depth scale이 크게 흔들린다.
10 px	약 2.63 m	$\sigma_Z \approx 0.13$ m	중거리 feature. calibration 오차 영향도 같이 확인해야 한다.
20 px	약 1.31 m	$\sigma_Z \approx 0.03$ m	가까운 feature는 depth가 안정적이지만 occlusion과 matching 실패를 본다.

실제 코드

읽는 포인트: left image에서 찾은 feature를 right image로 다시 KLT 추적해서 stereo correspondence를 만든다.

feature_tracker.cpp:216-221

```

216         cv::calcOpticalFlowPyrLK(cur_img, rightImg, cur_pts, cur_right_pts, status, err, cv::Size(21,
217             21), 3);
218         // reverse check cur right ---- cur left
219         if (FLOW_BACK)
220         {

```

```

220         cv::calcOpticalFlowPyrLK(rightImg, cur_img, cur_right_pts, reverseLeftPts,
            statusRightLeft, err, cv::Size(21, 21), 3);
221         for(size_t i = 0; i < status.size(); i++)

```

코드 분석

- temporal tracking이 아니라 left-right stereo matching이다. 같은 시간의 왼쪽 feature를 오른쪽 이미지에서 찾는다.
- 이 대응점으로 disparity $d = u_L - u_R$ 가 만들어지고, 결국 $Z = f_x b / d$ 의 depth scale이 정해진다.
- 수중에서는 굴절과 카메라 하우징 때문에 공기 중 calibration을 그대로 쓰면 이 matching이 맞아 보여도 metric depth가 틀릴 수 있다.

feature_manager.cpp:198-212

```

198 void FeatureManager::triangulatePoint(Eigen::Matrix<double, 3, 4> &Pose0, Eigen::Matrix<double, 3, 4> &
    Pose1,
199         Eigen::Vector2d &point0, Eigen::Vector2d &point1, Eigen::Vector3d &point_3d)
200 {
201     Eigen::Matrix4d design_matrix = Eigen::Matrix4d::Zero();
202     design_matrix.row(0) = point0[0] * Pose0.row(2) - Pose0.row(0);
203     design_matrix.row(1) = point0[1] * Pose0.row(2) - Pose0.row(1);
204     design_matrix.row(2) = point1[0] * Pose1.row(2) - Pose1.row(0);
205     design_matrix.row(3) = point1[1] * Pose1.row(2) - Pose1.row(1);
206     Eigen::Vector4d triangulated_point;
207     triangulated_point =
208         design_matrix.jacobiSvd(Eigen::ComputeFullV).matrixV().rightCols<1>();
209     point_3d(0) = triangulated_point(0) / triangulated_point(3);
210     point_3d(1) = triangulated_point(1) / triangulated_point(3);
211     point_3d(2) = triangulated_point(2) / triangulated_point(3);
212 }

```

코드 분석

- 여러 camera pose와 image observation으로 선형 행렬 A 를 만들고 SVD로 homogeneous 3D point를 구한다. 문서의 $AX = 0$ 에 해당한다.
- 이 코드는 feature가 어느 3D 위치에 있는지 초기값을 만든다. 초기 depth가 나쁘면 inverse depth 최적화도 잘못된 곳에서 시작한다.
- 반복 타일에서는 2D 대응이 잘못되어도 삼각측량 자체는 어떤 3D 점을 만들어낸다. 그래서 triangulation 전 correspondence filtering이 필요하다.

feature_manager.cpp:302-340

```

302 void FeatureManager::triangulate(int frameCnt, Vector3d Ps[], Matrix3d Rs[], Vector3d tic[], Matrix3d ric
    [])
303 {
304     for (auto &it_per_id : feature)
305     {
306         if (it_per_id.estimated_depth > 0)
307             continue;
308
309         if (STEREO && it_per_id.feature_per_frame[0].is_stereo)
310         {
311             int imu_i = it_per_id.start_frame;
312             Eigen::Matrix<double, 3, 4> leftPose;
313             Eigen::Vector3d t0 = Ps[imu_i] + Rs[imu_i] * tic[0];
314             Eigen::Matrix3d R0 = Rs[imu_i] * ric[0];
315             leftPose.leftCols<3>() = R0.transpose();
316             leftPose.rightCols<1>() = -R0.transpose() * t0;
317             //cout << "left pose " << leftPose << endl;
318
319             Eigen::Matrix<double, 3, 4> rightPose;
320             Eigen::Vector3d t1 = Ps[imu_i] + Rs[imu_i] * tic[1];
321             Eigen::Matrix3d R1 = Rs[imu_i] * ric[1];
322             rightPose.leftCols<3>() = R1.transpose();
323             rightPose.rightCols<1>() = -R1.transpose() * t1;
324             //cout << "right pose " << rightPose << endl;
325
326             Eigen::Vector2d point0, point1;
327             Eigen::Vector3d point3d;
328             point0 = it_per_id.feature_per_frame[0].point.head(2);
329             point1 = it_per_id.feature_per_frame[0].pointRight.head(2);

```

```

330         //cout << "point0 " << point0.transpose() << endl;
331         //cout << "point1 " << point1.transpose() << endl;
332
333         triangulatePoint(leftPose, rightPose, point0, point1, point3d);
334         Eigen::Vector3d localPoint;
335         localPoint = leftPose.leftCols<3>() * point3d + leftPose.rightCols<1>();
336         double depth = localPoint.z();
337         if (depth > 0)
338             it_per_id.estimated_depth = depth;
339         else
340             it_per_id.estimated_depth = INIT_DEPTH;

```

코드 분석

- stereo observation 이 있으면 두 camera 사이의 baseline으로 depth를 초기화한다. 이 값이 backend의 $\lambda = 1/Z$ 초기 값이 된다.
- depth가 음수이거나 너무 불안정하면 feature는 좋은 landmark가 아니다. 수중에서는 먼 벽면 feature가 작은 disparity 때문에 특히 흔들린다.
- 여기서 쓰는 camera extrinsic과 projection matrix가 실제 수중 stereo calibration과 맞아야 한다. 그렇지 않으면 RViz 경로 scale이 부풀거나 줄어든다.

9 4단계: Feature manager와 keyframe/parallax

핵심 정의: Parallax와 keyframe

Parallax는 같은 3D 점이 서로 다른 frame에서 얼마나 다른 위치로 보이는지를 뜻한다. parallax가 충분해야 triangulation으로 depth를 안정적으로 만들 수 있다. Keyframe은 backend window 안에 남겨둘 만큼 geometry 정보가 좋은 frame이다.

간단 예시: 앞으로 이동 vs 벽을 정면으로 보기

UUV가 옆으로 움직이며 부표를 보면 feature 위치가 크게 바뀌어 parallax가 생긴다. 반대로 타일 벽을 거의 정면으로 보며 천천히 전진하면 이미지 변화가 작아 parallax가 부족할 수 있다. 이 경우 feature 수는 많아도 depth와 pose constraint는 약할 수 있다.

수식

VINS는 같은 feature가 frame 사이에서 얼마나 이동했는지 평균 parallax를 계산한다.

$$\bar{p} = \frac{1}{N} \sum_{j=1}^N \left\| \mathbf{z}_{k-1}^j - \mathbf{z}_{k-2}^j \right\|$$

각 항의 의미

항	의미
$\bar{p}$	평균 parallax. keyframe 판단의 핵심 값.
N	parallax 계산에 사용된 공통 feature track 수.
$\mathbf{z}_{k-1}^j, \mathbf{z}_{k-2}^j$	같은 feature j 의 normalized image coordinate.
MIN_PARALLAX	keyframe으로 인정할 최소 평균 parallax. $\text{keyframe_parallax} / \text{FOCAL_LENGTH}$.

threshold와 keyframe 수

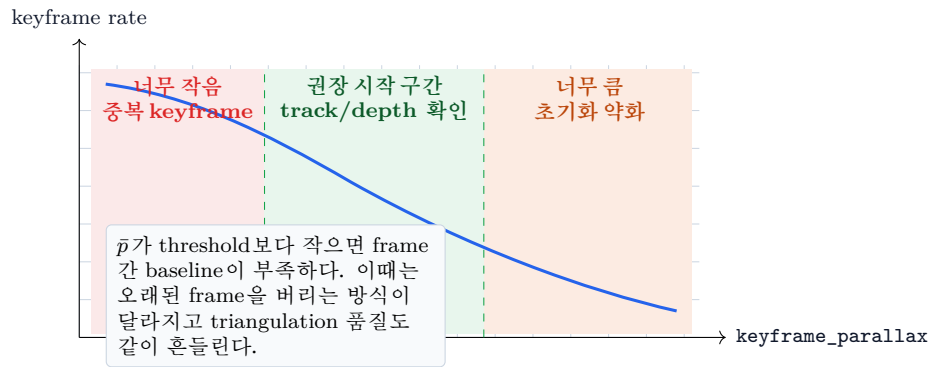


그림 6. keyframe_parallax는 속도 파라미터가 아니라 geometry 품질 파라미터다. 작게 잡으면 계산량이 늘고, 크게 잡으면 depth/초기화가 약해진다.

파라미터와 영향

파라미터/조건	수식항 관계	작게/부족하면	크게/많으면
keyframe_parallax	$\bar{p} \geq \text{MIN_PARALLAX threshold}$	keyframe이 많아져 계산량 증가, 비슷한 frame이 window를 채움.	keyframe이 적어져 triangulation/초기화가 약해질 수 있음.
last_track_num	공통 feature 수 N	parallax 통계 불안정, tracking 품질 나쁨.	통계 안정. 단, 반복 타일 outlier인지 확인 필요.
long_track_num	feature lifetime	track이 짧아 pose constraint가 약함.	장기 constraint에 유리. 잘못 붙은 track이면 위험.
new_feature_num	새 feature 비율	너무 적으면 저 텍스처/blur.	너무 많으면 기존 track이 자주 끊긴다는 뜻.

실제 코드

읽는 포인트: feature manager가 새 frame을 keyframe으로 볼지 판단하고, 그 결과가 marginalization 방향으로 이어진다.

estimator.cpp:270-319

```

270 void Estimator::processMeasurements()
271 {
272     while (1)
273     {
274         //printf("process measurments\n");
275         pair<double, map<int, vector<pair<int, Eigen::Matrix<double, 7, 1> > > > > feature;
276         vector<pair<double, Eigen::Vector3d>> accVector, gyrVector;
277         if(!featureBuf.empty())
278         {
279             feature = featureBuf.front();
280             curTime = feature.first + td;
281             while(1)
282             {
283                 if ((USE_IMU || IMUAvailable(feature.first + td)))
284                     break;
285                 else
286                 {
287                     printf("wait for imu ... \n");
288                     if (! MULTIPLE_THREAD)
289                         return;
290                     std::chrono::milliseconds dura(5);
291                     std::this_thread::sleep_for(dura);
292                 }
293             }
294             mBuf.lock();
295             if(USE_IMU)
296                 getIMUInterval(prevTime, curTime, accVector, gyrVector);
297
298             featureBuf.pop();
299             mBuf.unlock();
300
301             if(USE_IMU)
302             {
303                 if(!initFirstPoseFlag)
304                     initFirstIMUPose(accVector);
305                 for(size_t i = 0; i < accVector.size(); i++)
306                 {
307                     double dt;
308                     if(i == 0)
309                         dt = accVector[i].first - prevTime;
310                     else if (i == accVector.size() - 1)
311                         dt = curTime - accVector[i - 1].first;
312                     else
313                         dt = accVector[i].first - accVector[i - 1].first;

```

```

314         processIMU(accVector[i].first, dt, accVector[i].second, gyrVector[i].second);
315     }
316 }
317 mProcess.lock();
318 processImage(feature.second, feature.first);
319 prevTime = curTime;

```

코드 분석

- image measurement가 들어오면 feature manager에 추가되고, 현재 frame을 keyframe으로 유지할지 판단한다.
- 이 판단은 단순 frame count가 아니라 parallax와 track 상태를 본다. 그래서 느리게 움직이거나 벽면만 보는 구간에서는 keyframe 선택이 약해질 수 있다.
- 수중 UUV에서 같은 타일 벽을 평행하게 보면 parallax가 작아 depth가 약해진다. 이때 path가 정지하거나 갑자기 될 수 있다.

estimator.cpp:411-426

```

411 void Estimator::processImage(const map<int, vector<pair<int, Eigen::Matrix<double, 7, 1>>>> &image, const
    double header)
412 {
413     ROS_DEBUG("new image coming -----");
414     ROS_DEBUG("Adding feature points %lu", image.size());
415     if (f_manager.addFeatureCheckParallax(frame_count, image, td))
416     {
417         marginalization_flag = MARGIN_OLD;
418         //printf("keyframe\n");
419     }
420     else
421     {
422         marginalization_flag = MARGIN_SECOND_NEW;
423         //printf("non-keyframe\n");
424     }
425
426     ROS_DEBUG("%s", marginalization_flag ? "Non-keyframe" : "Keyframe");

```

코드 분석

- MARGIN_OLD와 MARGIN_SECOND_NEW는 sliding window에서 어떤 frame을 제거할지 정한다.
- keyframe으로 볼 만큼 parallax가 있으면 오래된 frame을 밀어내고, parallax가 부족하면 최근 frame 쪽을 제거해 window의 geometry를 유지하려 한다.
- keyframe_parallax를 너무 작게 잡으면 비슷한 frame이 window를 채우고, 너무 크게 잡으면 필요한 keyframe이 남지 않는다.

feature_manager.cpp:52-118

```

52 bool FeatureManager::addFeatureCheckParallax(int frame_count, const map<int, vector<pair<int, Eigen::
    Matrix<double, 7, 1>>>> &image, double td)
53 {
54     ROS_DEBUG("input feature: %d", (int)image.size());
55     ROS_DEBUG("num of feature: %d", getFeatureCount());
56     double parallax_sum = 0;
57     int parallax_num = 0;
58     last_track_num = 0;
59     last_average_parallax = 0;
60     new_feature_num = 0;
61     long_track_num = 0;
62     for (auto &id_pts : image)
63     {
64         FeaturePerFrame f_per_fra(id_pts.second[0].second, td);
65         assert(id_pts.second[0].first == 0);
66         if(id_pts.second.size() == 2)
67         {
68             f_per_fra.rightObservation(id_pts.second[1].second);
69             assert(id_pts.second[1].first == 1);
70         }
71
72         int feature_id = id_pts.first;
73         auto it = find_if(feature.begin(), feature.end(), [feature_id](const FeaturePerId &it)
74             {
75                 return it.feature_id == feature_id;

```

```

76         });
77
78         if (it == feature.end())
79         {
80             feature.push_back(FeaturePerId(feature_id, frame_count));
81             feature.back().feature_per_frame.push_back(f_per_fra);
82             new_feature_num++;
83         }
84         else if (it->feature_id == feature_id)
85         {
86             it->feature_per_frame.push_back(f_per_fra);
87             last_track_num++;
88             if( it-> feature_per_frame.size() >= 4)
89                 long_track_num++;
90         }
91     }
92
93     //if (frame_count < 2 || last_track_num < 20)
94     //if (frame_count < 2 || last_track_num < 20 || new_feature_num > 0.5 * last_track_num)
95     if (frame_count < 2 || last_track_num < 20 || long_track_num < 40 || new_feature_num > 0.5 *
96         last_track_num)
97         return true;
98
99     for (auto &it_per_id : feature)
100     {
101         if (it_per_id.start_frame <= frame_count - 2 &&
102             it_per_id.start_frame + int(it_per_id.feature_per_frame.size()) - 1 >= frame_count - 1)
103         {
104             parallax_sum += compensatedParallax2(it_per_id, frame_count);
105             parallax_num++;
106         }
107     }
108
109     if (parallax_num == 0)
110     {
111         return true;
112     }
113     else
114     {
115         ROS_DEBUG("parallax_sum: %lf, parallax_num: %d", parallax_sum, parallax_num);
116         ROS_DEBUG("current parallax: %lf", parallax_sum / parallax_num * FOCAL_LENGTH);
117         last_average_parallax = parallax_sum / parallax_num * FOCAL_LENGTH;
118         return parallax_sum / parallax_num >= MIN_PARALLAX;
119     }

```

코드 분석

- 새 feature, 오래 추적된 feature, 공통 track 수를 세면서 frontend 품질을 요약한다. 이 값들은 수중 실험 로그에서 반드시 봐야 하는 진단 지표다.
- 평균 parallax는 normalized coordinate 기준이다. pixel threshold와 focal length가 연결되므로, resize를 바꾸면 같은 threshold가 같은 의미가 아니다.
- 반복 패턴에서 잘못 붙은 long track은 오히려 위험하다. track이 길다는 사실만 보지 말고 forward-backward error와 reprojection residual을 같이 봐야 한다.

10 5단계: IMU preintegration

핵심 정의: IMU preintegration

IMU는 camera frame보다 훨씬 빠르게 accelerometer와 gyroscope 값을 낸다. Preintegration은 두 camera frame 사이에 들어온 많은 IMU sample을 하나의 상대 motion factor로 압축하는 과정이다. 이렇게 해야 backend가 모든 IMU sample을 직접 최적화하지 않고도 motion prior를 사용할 수 있다.

간단 예시: 10 Hz camera와 200 Hz IMU

camera가 0.1초마다 한 장씩 들어오고 IMU가 0.005초마다 들어오면, camera frame 두 개 사이에 IMU sample이 약 20개 있다. preintegration은 이 20개를 모아 “이 0.1초 동안 대략 이만큼 회전하고 이동했다”는 하나의 residual로 만든다.

수식

camera frame i 와 j 사이의 많은 IMU sample을 하나의 factor로 압축한다.

$$\Delta R_{ij} \approx \prod_{k=i}^{j-1} \exp((\omega_k - \mathbf{b}_g) \Delta t)$$

$$\Delta \mathbf{v}_{ij} \approx \sum_{k=i}^{j-1} R_i^\top (R_k(\mathbf{a}_k - \mathbf{b}_a) - \mathbf{g}) \Delta t$$

$$\mathbf{x}_i = [\mathbf{p}_i, \mathbf{q}_i, \mathbf{v}_i, \mathbf{b}_{a_i}, \mathbf{b}_{g_i}]$$

각 항의 의미

항	의미
$\Delta R_{ij}, \Delta \mathbf{v}_{ij}$	IMU가 예측한 frame $i \rightarrow j$ 상대 회전/속도 변화.
$\omega_k, \mathbf{a}_k$	gyro와 accelerometer measurement.
$\mathbf{b}_g, \mathbf{b}_a$	gyro/accelerometer bias.
Δt	IMU sample 간 시간 간격. timestamp 품질에 민감.
$\mathbf{g}$	gravity vector. 초기 alignment와 velocity 추정에 중요.
$\mathbf{x}_i$	pose, velocity, IMU bias를 포함한 VIO 상태.

파라미터와 영향

파라미터	수식항 관계	작게 하면	크게 하면
acc_n	accel residual covariance	IMU 가속도를 과신. 영상과 충돌 가능.	IMU를 덜 믿어 feature 부족 구간 보완 약화.
gyr_n	gyro residual covariance	회전을 과신. gyro noise/bias에 취약.	빠른 회전에서 vision 의존도 증가.
acc_w	accel bias random walk	bias drift를 못 따라감.	bias가 motion까지 흡수할 수 있음.
gyr_w	gyro bias random walk	yaw/rotation drift를 못 따라감.	bias 추정이 과하게 흔들림.
td	camera-IMU time offset	대소보다 정확성이 중요. 틀리면 visual/IMU residual 충돌.	동일. 빠른 motion에서 더 민감.

현재 MP4 실험은 imu: 0이라 이 factor가 빠져 있다. UUV VIO-SLAM에서는 imu: 1, camera-IMU extrinsic, time offset, noise parameter를 반드시 맞춰야 한다.

실제 코드

읽는 포인트: IMU sample이 들어올 때마다 preintegration과 임시 propagation이 동시에 갱신된다.

estimator.cpp:375-405

```

375 void Estimator::processIMU(double t, double dt, const Vector3d &linear_acceleration, const Vector3d &
    angular_velocity)
376 {
377     if (!first_imu)
378     {
379         first_imu = true;
380         acc_0 = linear_acceleration;

```

```

381     gyr_0 = angular_velocity;
382 }
383
384 if (!pre_integrations[frame_count])
385 {
386     pre_integrations[frame_count] = new IntegrationBase{acc_0, gyr_0, Bas[frame_count], Bgs[
frame_count]};
387 }
388 if (frame_count != 0)
389 {
390     pre_integrations[frame_count]->push_back(dt, linear_acceleration, angular_velocity);
391     //if(solver_flag != NON_LINEAR)
392         tmp_pre_integration->push_back(dt, linear_acceleration, angular_velocity);
393
394     dt_buf[frame_count].push_back(dt);
395     linear_acceleration_buf[frame_count].push_back(linear_acceleration);
396     angular_velocity_buf[frame_count].push_back(angular_velocity);
397
398     int j = frame_count;
399     Vector3d un_acc_0 = Rs[j] * (acc_0 - Bas[j]) - g;
400     Vector3d un_gyr = 0.5 * (gyr_0 + angular_velocity) - Bgs[j];
401     Rs[j] *= Utility::deltaQ(un_gyr * dt).toRotationMatrix();
402     Vector3d un_acc_1 = Rs[j] * (linear_acceleration - Bas[j]) - g;
403     Vector3d un_acc = 0.5 * (un_acc_0 + un_acc_1);
404     Ps[j] += dt * Vs[j] + 0.5 * dt * dt * un_acc;
405     Vs[j] += dt * un_acc;

```

코드 분석

- IMU sample 사이의 Δt 를 계산하고, acceleration/gyro를 preintegration에 누적한다.
- 동시에 현재 pose/velocity를 임시로 propagation한다. 이 값은 image가 들어오기 전 motion prior처럼 쓰인다.
- Δt 가 틀리면 $\Delta R, \Delta v, \Delta p$ 가 모두 틀어진다. 수중 로봇에서 ROS timestamp와 camera timestamp sync가 중요한 이유다.

integration_base.h:139-194

```

139 void propagate(double _dt, const Eigen::Vector3d &acc_1, const Eigen::Vector3d &gyr_1)
140 {
141     dt = _dt;
142     acc_1 = _acc_1;
143     gyr_1 = _gyr_1;
144     Vector3d result_delta_p;
145     Quaterniond result_delta_q;
146     Vector3d result_delta_v;
147     Vector3d result_linearized_ba;
148     Vector3d result_linearized_bg;
149
150     midPointIntegration(_dt, acc_0, gyr_0, _acc_1, _gyr_1, delta_p, delta_q, delta_v,
151         linearized_ba, linearized_bg,
152         result_delta_p, result_delta_q, result_delta_v,
153         result_linearized_ba, result_linearized_bg, 1);
154
155     //checkJacobian(_dt, acc_0, gyr_0, acc_1, gyr_1, delta_p, delta_q, delta_v,
156     //    linearized_ba, linearized_bg);
157     delta_p = result_delta_p;
158     delta_q = result_delta_q;
159     delta_v = result_delta_v;
160     linearized_ba = result_linearized_ba;
161     linearized_bg = result_linearized_bg;
162     delta_q.normalize();
163     sum_dt += dt;
164     acc_0 = acc_1;
165     gyr_0 = gyr_1;
166 }
167
168 Eigen::Matrix<double, 15, 1> evaluate(const Eigen::Vector3d &Pi, const Eigen::Quaterniond &Qi, const
Eigen::Vector3d &Vi, const Eigen::Vector3d &Bai, const Eigen::Vector3d &Bgi,
169     const Eigen::Vector3d &Pj, const Eigen::Quaterniond &Qj, const
Eigen::Vector3d &Vj, const Eigen::Vector3d &Baj, const Eigen::Vector3d &Bgj)
170 {
171     Eigen::Matrix<double, 15, 1> residuals;
172
173

```

```

174 Eigen::Matrix3d dp_dba = jacobian.block<3, 3>(0_P, 0_BA);
175 Eigen::Matrix3d dp_dbg = jacobian.block<3, 3>(0_P, 0_BG);
176
177 Eigen::Matrix3d dq_dbg = jacobian.block<3, 3>(0_R, 0_BG);
178
179 Eigen::Matrix3d dv_dba = jacobian.block<3, 3>(0_V, 0_BA);
180 Eigen::Matrix3d dv_dbg = jacobian.block<3, 3>(0_V, 0_BG);
181
182 Eigen::Vector3d dba = Bai - linearized_ba;
183 Eigen::Vector3d dbg = Bgi - linearized_bg;
184
185 Eigen::Quaterniond corrected_delta_q = delta_q * Utility::deltaQ(dq_dbg * dbg);
186 Eigen::Vector3d corrected_delta_v = delta_v + dv_dba * dba + dv_dbg * dbg;
187 Eigen::Vector3d corrected_delta_p = delta_p + dp_dba * dba + dp_dbg * dbg;
188
189 residuals.block<3, 1>(0_P, 0) = Qi.inverse() * (0.5 * G * sum_dt * sum_dt + Pj - Pi - Vi * sum_dt
190 ) - corrected_delta_p;
191 residuals.block<3, 1>(0_R, 0) = 2 * (corrected_delta_q.inverse() * (Qi.inverse() * Qj)).vec();
192 residuals.block<3, 1>(0_V, 0) = Qi.inverse() * (G * sum_dt + Vj - Vi) - corrected_delta_v;
193 residuals.block<3, 1>(0_BA, 0) = Baj - Bai;
194 residuals.block<3, 1>(0_BG, 0) = Bgj - Bgi;
195 return residuals;

```

코드 분석

- 이 코드는 preintegration residual과 Jacobian을 만든다. 즉 IMU raw sample 여러 개를 하나의 factor r_{imu} 로 압축한다.
- bias $\mathbf{b}_a, \mathbf{b}_g$ 에 대한 보정항이 들어간다. bias를 상태로 추정하기 때문에 IMU가 시간이 지나며 조금씩 변하는 것을 모델링할 수 있다.
- $\mathbf{acc}_n / \mathbf{gyr}_n$ 은 residual covariance에 연결된다. 너무 작게 잡으면 IMU를 과신하고, 너무 크게 잡으면 feature가 부족한 구간에서 IMU 도움을 못 받는다.

imu_factor.h:21-75

```

21 class IMUFactor : public ceres::SizedCostFunction<15, 7, 9, 7, 9>
22 {
23 public:
24     IMUFactor() = delete;
25     IMUFactor(IntegrationBase* _pre_integration):pre_integration(_pre_integration)
26     {
27     }
28     virtual bool Evaluate(double const* const* parameters, double* residuals, double** jacobians) const
29     {
30
31         Eigen::Vector3d Pi(parameters[0][0], parameters[0][1], parameters[0][2]);
32         Eigen::Quaterniond Qi(parameters[0][6], parameters[0][3], parameters[0][4], parameters[0][5]);
33
34         Eigen::Vector3d Vi(parameters[1][0], parameters[1][1], parameters[1][2]);
35         Eigen::Vector3d Bai(parameters[1][3], parameters[1][4], parameters[1][5]);
36         Eigen::Vector3d Bgi(parameters[1][6], parameters[1][7], parameters[1][8]);
37
38         Eigen::Vector3d Pj(parameters[2][0], parameters[2][1], parameters[2][2]);
39         Eigen::Quaterniond Qj(parameters[2][6], parameters[2][3], parameters[2][4], parameters[2][5]);
40
41         Eigen::Vector3d Vj(parameters[3][0], parameters[3][1], parameters[3][2]);
42         Eigen::Vector3d Baj(parameters[3][3], parameters[3][4], parameters[3][5]);
43         Eigen::Vector3d Bgj(parameters[3][6], parameters[3][7], parameters[3][8]);
44
45         //Eigen::Matrix<double, 15, 15> Fd;
46         //Eigen::Matrix<double, 15, 12> Gd;
47
48         //Eigen::Vector3d pPj = Pi + Vi * sum_t - 0.5 * g * sum_t * sum_t + corrected_delta_p;
49         //Eigen::Quaterniond pQj = Qi * delta_q;
50         //Eigen::Vector3d pVj = Vi - g * sum_t + corrected_delta_v;
51         //Eigen::Vector3d pBaj = Bai;
52         //Eigen::Vector3d pBgj = Bgi;
53
54         //Vi + Qi * delta_v - g * sum_dt = Vj;
55         //Qi * delta_q = Qj;
56
57         //delta_p = Qi.inverse() * (0.5 * g * sum_dt * sum_dt + Pj - Pi);
58         //delta_v = Qi.inverse() * (g * sum_dt + Vj - Vi);

```

```

59 //delta_q = Qi.inverse() * Qj;
60
61 #if 0
62     if ((Bai - pre_integration->linearized_ba).norm() > 0.10 ||
63         (Bgi - pre_integration->linearized_bg).norm() > 0.01)
64     {
65         pre_integration->repropagate(Bai, Bgi);
66     }
67 #endif
68
69     Eigen::Map<Eigen::Matrix<double, 15, 1>> residual(residuals);
70     residual = pre_integration->evaluate(Pi, Qi, Vi, Bai, Bgi,
71                                         Pj, Qj, Vj, Baj, Bgj);
72
73     Eigen::Matrix<double, 15, 15> sqrt_info = Eigen::LLT<Eigen::Matrix<double, 15, 15>>(<
pre_integration->covariance.inverse()).matrixL().transpose();
74     //sqrt_info.setIdentity();
75     residual = sqrt_info * residual;

```

코드 분석

- Ceres가 호출하는 IMU residual factor다. 두 frame의 pose, velocity, bias를 받아 preintegration 예측과 실제 상태 차이를 residual로 계산한다.
- visual factor가 image plane error를 줄인다면, IMU factor는 frame 사이 motion이 accelerometer/gyro와 일치하도록 묶는다.
- UUV에서는 회전이 빠르거나 영상 feature가 끊기는 순간에 이 factor가 중요하다. 단, extrinsic/time offset 이 틀리면 IMU가 오히려 pose를 잘못 끌고 간다.

11 6단계: Initialization

핵심 정의: Initialization

Initialization은 nonlinear optimization을 시작하기 위한 첫 pose, depth, velocity, bias 추정값을 만드는 과정이다. 초기값이 너무 틀리면 Ceres가 아무리 최적화해도 잘못된 local minimum에 빠질 수 있다.

간단 예시: PnP가 하는 일

이미 depth를 아는 feature 10개가 있고, 현재 이미지에서 그 feature들이 보이는 2D 위치를 안다고 하자. PnP는 “이 3D 점들이 현재 이미지에 이렇게 보이려면 camera pose가 어디여야 하는가?”를 푸는 문제다.

수식

stereo depth로 얻은 3D point와 현재 frame의 2D 관측을 이용해 PnP로 초기 pose를 만든다.

$$\min_{R,t} \sum_j \|\pi(R\mathbf{X}_j + t) - \mathbf{z}_j\|^2$$

각 항의 의미

항	의미
R, t	현재 frame pose. PnP가 추정하는 회전과 이동.
$\mathbf{X}_j$	이미 depth가 있는 3D feature point.
$\mathbf{z}_j$	현재 frame의 2D normalized feature coordinate.
$\pi(\cdot)$	3D point를 image plane으로 projection하는 함수.
$\pi(R\mathbf{X}_j + t) - \mathbf{z}_j$	reprojection error.

파라미터와 영향

항목	수식항 관계	부족/작으면	과함/크면
3D-2D correspondence 수	$\sum_j$ 의 항 개수	4개 미만이면 PnP 실패. 적을수록 불안정.	많으면 안정적이나 outlier가 섞이면 pose가 망가짐.
depth 품질	X_j	scale과 pose 초기값이 틀어짐.	정확성이 중요. 값 자체가 많아도 calibration이 틀리면 의미 없음.
keyframe_parallax	좋은 X_j 생성 조건	작은 motion에서도 초기화해 depth가 약함.	초기화가 늦거나 실패할 수 있음.

실제 코드

읽는 포인트: 이미 depth가 있는 feature를 3D point로 쓰고, 현재 frame의 2D 관측과 맞춰 pose 초기값을 만든다.

feature_manager.cpp:215-238

```

215 bool FeatureManager::solvePoseByPnP(Eigen::Matrix3d &R, Eigen::Vector3d &P,
216                                     vector<cv::Point2f> &pts2D, vector<cv::Point3f> &pts3D)
217 {
218     Eigen::Matrix3d R_initial;
219     Eigen::Vector3d P_initial;
220
221     // w_T_cam ---> cam_T_w
222     R_initial = R.inverse();
223     P_initial = -(R_initial * P);
224
225     //printf("pnp size %d \n", (int)pts2D.size());
226     if ((int)pts2D.size()) < 4)
227     {
228         printf("feature tracking not enough, please slowly move you device! \n");
229         return false;
230     }
231     cv::Mat r, rvec, t, D, tmp_r;
232     cv::eigen2cv(R_initial, tmp_r);
233     cv::Rodrigues(tmp_r, rvec);
234     cv::eigen2cv(P_initial, t);
235     cv::Mat K = (cv::Mat_<double>(3, 3) << 1, 0, 0, 0, 1, 0, 0, 0, 1);
236     bool pnp_succ;
237     pnp_succ = cv::solvePnP(pts3D, pts2D, K, D, rvec, t, 1);
238     //pnp_succ = solvePnPransac(pts3D, pts2D, K, D, rvec, t, true, 100, 8.0 / focalLength, 0.99, inliers);

```

코드 분석

- solvePnP는 depth가 있는 3D landmark와 현재 2D feature를 맞춰 camera pose 초기값을 계산한다.
- 이 pose는 Ceres 최적화의 출발점이다. 초기값이 너무 나쁘면 nonlinear optimization이 올바른 최소점으로 가지 못한다.
- 수중에서는 depth가 불안정한 feature를 PnP에 넣는 것이 위험하다. 특히 먼 타일 벽 feature는 disparity error가 pose 초기값을 크게 혼든다.

feature_manager.cpp:259-300

```

259 void FeatureManager::initFramePoseByPnP(int frameCnt, Vector3d Ps[], Matrix3d Rs[], Vector3d tic[],
260                                         Matrix3d ric[])
261 {
262     if(frameCnt > 0)
263     {
264         vector<cv::Point2f> pts2D;

```

```

265     vector<cv::Point3f> pts3D;
266     for (auto &it_per_id : feature)
267     {
268         if (it_per_id.estimated_depth > 0)
269         {
270             int index = frameCnt - it_per_id.start_frame;
271             if((int)it_per_id.feature_per_frame.size() >= index + 1)
272             {
273                 Vector3d ptsInCam = ric[0] * (it_per_id.feature_per_frame[0].point * it_per_id.
estimated_depth) + tic[0];
274                 Vector3d ptsInWorld = Rs[it_per_id.start_frame] * ptsInCam + Ps[it_per_id.start_frame
];
275
276                 cv::Point3f point3d(ptsInWorld.x(), ptsInWorld.y(), ptsInWorld.z());
277                 cv::Point2f point2d(it_per_id.feature_per_frame[index].point.x(), it_per_id.
feature_per_frame[index].point.y());
278                 pts3D.push_back(point3d);
279                 pts2D.push_back(point2d);
280             }
281         }
282     }
283     Eigen::Matrix3d RCam;
284     Eigen::Vector3d PCam;
285     // trans to w_T_cam
286     RCam = Rs[frameCnt - 1] * ric[0];
287     PCam = Rs[frameCnt - 1] * tic[0] + Ps[frameCnt - 1];
288
289     if(solvePoseByPnP(RCam, PCam, pts2D, pts3D))
290     {
291         // trans to w_T_imu
292         Rs[frameCnt] = RCam * ric[0].transpose();
293         Ps[frameCnt] = -RCam * ric[0].transpose() * tic[0] + PCam;
294
295         Eigen::Quaterniond Q(Rs[frameCnt]);
296         //cout << "frameCnt: " << frameCnt << " pnp Q " << Q.w() << " " << Q.vec().transpose() <<
endl;
297         //cout << "frameCnt: " << frameCnt << " pnp P " << Ps[frameCnt].transpose() << endl;
298     }
299 }
300 }

```

코드 분석

- PnP에 넣을 3D-2D correspondence를 수집한다. feature가 여러 frame에서 관측되고 depth가 있어야 후보가 된다.
- correspondence 개수가 적으면 PnP가 불안정하고, 많어도 outlier가 섞이면 pose가 틀어진다. 그래서 frontend filtering과 depth 검증이 선행 조건이다.
- 수영장 환경에서는 같은 타일 모서리가 서로 바뀌어 붙는 경우가 많다. 이 경우 correspondence 수는 충분해 보여도 실제 pose constraint는 틀릴 수 있다.

estimator.cpp:508-522

```

508     // stereo only initialization
509     if(STEREO && !USE_IMU)
510     {
511         f_manager.initFramePoseByPnP(frame_count, Ps, Rs, tic, ric);
512         f_manager.triangulate(frame_count, Ps, Rs, tic, ric);
513         optimization();
514
515         if(frame_count == WINDOW_SIZE)
516         {
517             optimization();
518             updateLatestStates();
519             solver_flag = NON_LINEAR;
520             slideWindow();
521             ROS_INFO("Initialization finish!");
522         }
523     }

```

코드 분석

- stereo-only 설정에서는 IMU alignment 없이 stereo depth와 PnP 기반으로 초기화가 진행된다.
- 지금 MP4 실험이 이 branch에 가깝다. 따라서 결과를 VIO라고 부르기도는 VINS-Fusion의 stereo visual odometry 실행으로 이해하는 것이 정확하다.

- 실제 대회용 UUV VIO로 가려면 이 초기화 뒤에 IMU factor가 켜지고, gravity/bias/extrinsic이 안정적으로 들어와야 한다.

12 7단계: Sliding-window factor graph optimization

핵심 정의: Factor graph optimization

Factor graph는 모르는 값인 상태 변수와, 그 값을 제한하는 residual factor를 그래프로 표현한 것이다. VINS backend는 최근 몇 개 frame만 window에 넣고, pose/velocity/bias/depth가 visual residual과 IMU residual을 동시에 잘 설명하도록 최적화한다.

간단 예시: 상태와 제약

상태 변수는 “frame 3의 pose”, “frame 4의 velocity”, “feature 21의 inverse depth” 같은 미지수다. 제약은 “이 feature가 이미지에서는 여기 보였다”, “IMU는 이 사이에 이만큼 회전했다고 말한다” 같은 관측이다. 최적화는 모든 제약의 오차가 전체적으로 작아지도록 상태를 조정한다.

수식

VINS-Fusion backend는 최근 window 안의 pose, velocity, bias, extrinsic, feature inverse depth를 동시에 최적화한다.

$$\mathcal{X} = \{T_{wb_0}, \dots, T_{wb_N}, \mathbf{v}_0, \dots, \mathbf{v}_N, \mathbf{b}_{a_0}, \dots, \mathbf{b}_{g_N}, T_{bc_0}, T_{bc_1}, \lambda_1, \dots, \lambda_M\}$$

$$\min_{\mathcal{X}} \sum_{\text{visual}} \rho(\|\mathbf{r}_{proj}\|^2) + \sum_{\text{imu}} \|\mathbf{r}_{imu}\|_{\Sigma^{-1}}^2 + \|\mathbf{r}_{prior}\|^2$$

visual residual은 다음 형태다.

$$\mathbf{r}_{ij} = \pi \left(T_{c_j b_j} T_{b_j w} T_{w b_i} T_{b_i c_i} \frac{1}{\lambda} \tilde{\mathbf{z}}_i \right) - \mathbf{z}_j$$

각 항의 의미

항	의미
$\mathcal{X}$	sliding window 안에서 동시에 추정하는 전체 상태.
T_{wb_i}	world에서 body/IMU frame i 의 pose.
$\mathbf{v}_i, \mathbf{b}_{a_i}, \mathbf{b}_{g_i}$	velocity, accel bias, gyro bias. IMU를 쓸 때 핵심.
T_{bc_0}, T_{bc_1}	body와 left/right camera extrinsic.
λ	inverse depth. depth Z 대신 $1/Z$ 를 최적화.
$\rho(\cdot)$	robust loss. outlier residual 영향을 줄인다. 코드에서는 Huber loss.
$\mathbf{r}_{proj}$	visual reprojection residual.
$\mathbf{r}_{imu}$	IMU preintegration residual.
$\mathbf{r}_{prior}$	marginalization으로 남은 prior residual.
Σ^{-1}	measurement covariance inverse. residual weight.

factor graph와 sparse Hessian

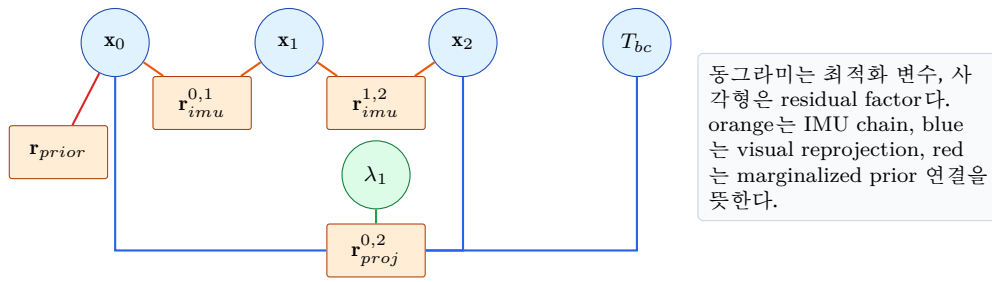


그림 7. VINS backend의 factor graph. 이 연결 구조가 아래 sparse Hessian의 block pattern으로 변환된다.

$$\begin{bmatrix} H_{x_0x_0} & H_{x_0x_1} & 0 & H_{x_0\lambda} \\ H_{x_1x_0} & H_{x_1x_1} & H_{x_1x_2} & 0 \\ 0 & H_{x_2x_1} & H_{x_2x_2} & H_{x_2\lambda} \\ H_{\lambda x_0} & 0 & H_{\lambda x_2} & H_{\lambda\lambda} \end{bmatrix} \delta \mathcal{X} = \mathbf{b}$$

파라미터와 영향

파라미터	수식항 관계	작게 하면	크게 하면
max_solver_time	Ceres solve 시간	realtime은 빠르지만 수렴 전 종료 가능.	정확도는 좋아질 수 있지만 frame rate 저하.
max_num_iterations	nonlinear iteration 수	초기값이 나쁘면 덜 수렴.	계산량 증가. outlier가 많으면 오래 돌려도 개선 제한.
MAX_CNT	visual residual 수	geometry constraint 부족.	residual과 outlier/계산량 증가.
Huber scale	$\rho(\cdot)$	정상 residual도 늘릴 수 있음.	outlier를 거의 그대로 믿을 수 있음.
WINDOW_SIZE	Hessian H 크기	빠르지만 history constraint 약함.	정확도 가능성은 높지만 계산량/marginalization 비용 증가.

backend 튜닝 판단 기준

현상	수식 관점	먼저 조정할 것
solver가 매번 시간 제한에 걸림	$H\delta x = b$ 를 충분히 풀기 전에 종료	MAX_CNT, frame rate, image resize를 낮춘 뒤 solver_time을 늘린다.
경로가 순간적으로 튕	큰 $\mathbf{r}_{proj}$ 또는 잘못된 λ 가 backend에 유입	feature outlier gate, stereo calibration, Huber scale을 먼저 확인한다.
느리게 계속 drift	local residual 만으로 global 위치를 고정하지 못함	loop, depth, marker, sonar factor 같은 global residual을 추가한다.
IMU와 vision이 서로 싸움	$\mathbf{r}_{imu}$ 와 $\mathbf{r}_{proj}$ 가 같은 motion을 설명하지 못함	td, extrinsic, acc_n/gyr_n, timestamp sync를 점검한다.

실제 코드

읽는 포인트: 상태 변수 block을 Ceres problem에 올리고, 이후 visual/IMU/prior residual이 붙는다.

estimator.cpp:1004-1060

```

1004 void Estimator::optimization()
1005 {
1006     TicToc t_whole, t_prepare;
1007     vector2double();
1008
1009     ceres::Problem problem;
1010     ceres::LossFunction *loss_function;
1011     //loss_function = NULL;
1012     loss_function = new ceres::HuberLoss(1.0);

```

```

1013 //loss_function = new ceres::CauchyLoss(1.0 / FOCAL_LENGTH);
1014 //ceres::LossFunction* loss_function = new ceres::HuberLoss(1.0);
1015 for (int i = 0; i < frame_count + 1; i++)
1016 {
1017     ceres::LocalParameterization *local_parameterization = new PoseLocalParameterization();
1018     problem.AddParameterBlock(para_Pose[i], SIZE_POSE, local_parameterization);
1019     if (USE_IMU)
1020         problem.AddParameterBlock(para_SpeedBias[i], SIZE_SPEEDBIAS);
1021 }
1022 if (!USE_IMU)
1023     problem.SetParameterBlockConstant(para_Pose[0]);
1024
1025 for (int i = 0; i < NUM_OF_CAM; i++)
1026 {
1027     ceres::LocalParameterization *local_parameterization = new PoseLocalParameterization();
1028     problem.AddParameterBlock(para_Ex_Pose[i], SIZE_POSE, local_parameterization);
1029     if ((ESTIMATE_EXTRINSIC && frame_count == WINDOW_SIZE && Vs[0].norm() > 0.2) || openExEstimation)
1030     {
1031         //ROS_INFO("estimate extinsic param");
1032         openExEstimation = 1;
1033     }
1034     else
1035     {
1036         //ROS_INFO("fix extinsic param");
1037         problem.SetParameterBlockConstant(para_Ex_Pose[i]);
1038     }
1039 }
1040 problem.AddParameterBlock(para_Td[0], 1);
1041
1042 if (!ESTIMATE_TD || Vs[0].norm() < 0.2)
1043     problem.SetParameterBlockConstant(para_Td[0]);
1044
1045 if (last_marginalization_info && last_marginalization_info->valid)
1046 {
1047     // construct new marginalization_factor
1048     MarginalizationFactor *marginalization_factor = new MarginalizationFactor(
1049         last_marginalization_info);
1050     problem.AddResidualBlock(marginalization_factor, NULL,
1051                             last_marginalization_parameter_blocks);
1052 }
1053 if (USE_IMU)
1054 {
1055     for (int i = 0; i < frame_count; i++)
1056     {
1057         int j = i + 1;
1058         if (pre_integrations[j]->sum_dt > 10.0)
1059             continue;
1060         IMUFactor* imu_factor = new IMUFactor(pre_integrations[j]);
1061         problem.AddResidualBlock(imu_factor, NULL, para_Pose[i], para_SpeedBias[i], para_Pose[j],
1062                                 para_SpeedBias[j]);

```

코드 분석

- pose, velocity, bias, extrinsic, inverse depth가 Ceres parameter block으로 등록된다. 문서의 $\mathcal{X}$ 가 실제 코드에서는 여러 double array block으로 나뉘어 들어간다.
- quaternion pose는 local parameterization을 써야 한다. 회전은 단순 4차원 벡터가 아니라 unit quaternion constraint를 유지해야 하기 때문이다.
- ESTIMATE_EXTRINSIC 여부에 따라 camera-IMU extrinsic이 고정되거나 최적화된다. 수중 하우징/마운트가 확실하지 않으면 이 설정이 path에 큰 영향을 준다.

estimator.cpp:1064-1104

```

1064 int f_m_cnt = 0;
1065 int feature_index = -1;
1066 for (auto &it_per_id : f_manager.feature)
1067 {
1068     it_per_id.used_num = it_per_id.feature_per_frame.size();
1069     if (it_per_id.used_num < 4)
1070         continue;
1071     ++feature_index;
1072 }
1073

```

```

1074     int imu_i = it_per_id.start_frame, imu_j = imu_i - 1;
1075
1076     Vector3d pts_i = it_per_id.feature_per_frame[0].point;
1077
1078     for (auto &it_per_frame : it_per_id.feature_per_frame)
1079     {
1080         imu_j++;
1081         if (imu_i != imu_j)
1082         {
1083             Vector3d pts_j = it_per_frame.point;
1084             ProjectionTwoFrameOneCamFactor *f_td = new ProjectionTwoFrameOneCamFactor(pts_i, pts_j,
1085                 it_per_id.feature_per_frame[0].velocity, it_per_frame.velocity,
1086                 it_per_id.feature_per_frame[0].cur_td,
1087                 it_per_frame.cur_td);
1088             problem.AddResidualBlock(f_td, loss_function, para_Pose[imu_i], para_Pose[imu_j],
1089                 para_Ex_Pose[0], para_Feature[feature_index], para_Td[0]);
1090         }
1091
1092         if(STEREO && it_per_frame.is_stereo)
1093         {
1094             Vector3d pts_j_right = it_per_frame.pointRight;
1095             if(imu_i != imu_j)
1096             {
1097                 ProjectionTwoFrameTwoCamFactor *f = new ProjectionTwoFrameTwoCamFactor(pts_i,
1098                     pts_j_right, it_per_id.feature_per_frame[0].velocity, it_per_frame.velocityRight,
1099                     it_per_id.feature_per_frame[0].cur_td,
1100                     it_per_frame.cur_td);
1101                 problem.AddResidualBlock(f, loss_function, para_Pose[imu_i], para_Pose[imu_j],
1102                     para_Ex_Pose[0], para_Ex_Pose[1], para_Feature[feature_index], para_Td[0]);
1103             }
1104             else
1105             {
1106                 ProjectionOneFrameTwoCamFactor *f = new ProjectionOneFrameTwoCamFactor(pts_i,
1107                     pts_j_right, it_per_id.feature_per_frame[0].velocity, it_per_frame.velocityRight,
1108                     it_per_id.feature_per_frame[0].cur_td,
1109                     it_per_frame.cur_td);
1110                 problem.AddResidualBlock(f, loss_function, para_Ex_Pose[0], para_Ex_Pose[1],
1111                     para_Feature[feature_index], para_Td[0]);
1112             }
1113         }
1114     }

```

코드 분석

- feature observation을 순회하면서 projection factor를 추가한다. 같은 landmark가 여러 frame/camera에 보이면 그만큼 residual edge가 생긴다.
- stereo인 경우 two-camera factor가 들어가므로 metric depth scale을 직접 constrain한다. 하지만 baseline/calibration이 틀리면 이 강한 constraint가 잘못된 scale을 강하게 밀어붙인다.
- outlier feature가 여기까지 들어오면 Ceres는 그 점도 설명하려고 pose와 depth를 움직인다. 그래서 frontend rejection이 backend 안정성의 핵심이다.

projectionTwoFrameTwoCamFactor.cpp:43-80

```

43 bool ProjectionTwoFrameTwoCamFactor::Evaluate(double const *parameters, double *residuals, double
    **jacobians) const
44 {
45     TicToc tic_toc;
46     Eigen::Vector3d Pi(parameters[0][0], parameters[0][1], parameters[0][2]);
47     Eigen::Quaterniond Qi(parameters[0][6], parameters[0][3], parameters[0][4], parameters[0][5]);
48
49     Eigen::Vector3d Pj(parameters[1][0], parameters[1][1], parameters[1][2]);
50     Eigen::Quaterniond Qj(parameters[1][6], parameters[1][3], parameters[1][4], parameters[1][5]);
51
52     Eigen::Vector3d tic(parameters[2][0], parameters[2][1], parameters[2][2]);
53     Eigen::Quaterniond qic(parameters[2][6], parameters[2][3], parameters[2][4], parameters[2][5]);
54
55     Eigen::Vector3d tic2(parameters[3][0], parameters[3][1], parameters[3][2]);
56     Eigen::Quaterniond qic2(parameters[3][6], parameters[3][3], parameters[3][4], parameters[3][5]);
57
58     double inv_dep_i = parameters[4][0];
59
60     double td = parameters[5][0];
61
62     Eigen::Vector3d pts_i_td, pts_j_td;

```

```

63     pts_i_td = pts_i - (td - td_i) * velocity_i;
64     pts_j_td = pts_j - (td - td_j) * velocity_j;
65
66     Eigen::Vector3d pts_camera_i = pts_i_td / inv_dep_i;
67     Eigen::Vector3d pts_imu_i = qic * pts_camera_i + tic;
68     Eigen::Vector3d pts_w = Qi * pts_imu_i + Pi;
69     Eigen::Vector3d pts_imu_j = Qj.inverse() * (pts_w - Pj);
70     Eigen::Vector3d pts_camera_j = qic2.inverse() * (pts_imu_j - tic2);
71     Eigen::Map<Eigen::Vector2d> residual(residuals);
72
73 #ifndef UNIT_SPHERE_ERROR
74     residual = tangent_base * (pts_camera_j.normalized() - pts_j_td.normalized());
75 #else
76     double dep_j = pts_camera_j.z();
77     residual = (pts_camera_j / dep_j).head<2>() - pts_j_td.head<2>();
78 #endif
79
80     residual = sqrt_info * residual;

```

코드 분석

- 이 factor는 한 feature를 기준 frame의 inverse depth로 3D화한 뒤, 다른 frame/다른 camera로 옮겨 projection error를 계산한다.
- 수식의 $T_{c_j b_j} T_{b_j w} T_{w b_i} T_{b_i c_i} \frac{1}{\lambda} \tilde{z}_i$ 가 코드의 좌표 변환 체인으로 구현되어 있다.
- residual은 image plane의 2D 오차다. 수중 굴절이나 잘못된 camera model은 이 residual을 체계적으로 키우므로, 단순 noise가 아니라 calibration/model 문제로 봐야 한다.

estimator.cpp:1113-1133

```

1113     ceres::Solver::Options options;
1114
1115     options.linear_solver_type = ceres::DENSE_SCHUR;
1116     //options.num_threads = 2;
1117     options.trust_region_strategy_type = ceres::DOGLLEG;
1118     options.max_num_iterations = NUM_ITERATIONS;
1119     //options.use_explicit_schur_complement = true;
1120     //options.minimizer_progress_to_stdout = true;
1121     //options.use_nonmonotonic_steps = true;
1122     if (marginalization_flag == MARGIN_OLD)
1123         options.max_solver_time_in_seconds = SOLVER_TIME * 4.0 / 5.0;
1124     else
1125         options.max_solver_time_in_seconds = SOLVER_TIME;
1126     TicToc t_solver;
1127     ceres::Solver::Summary summary;
1128     ceres::Solve(options, &problem, &summary);
1129     //cout << summary.BriefReport() << endl;
1130     ROS_DEBUG("Iterations : %d", static_cast<int>(summary.iterations.size()));
1131     //printf("solver costs: %f \n", t_solver.toc());
1132
1133     double2vector();

```

코드 분석

- 여기서 max_solver_time과 max_num_iterations가 실제 Ceres option으로 들어간다.
- 시간이 부족하면 optimization이 덜 수렴한 상태로 다음 frame으로 넘어간다. realtime은 좋아지지만 path가 덜 부드럽거나 drift가 커질 수 있다.
- 반대로 solver 시간을 늘려도 feature outlier가 많으면 더 오래 잘못된 문제를 풀 뿐이다. 속도 튜닝 전에는 residual 품질과 feature gate를 먼저 봐야 한다.

13 8단계: Marginalization과 sliding window

핵심 정의: Marginalization

Marginalization은 window에서 제거할 오래된 변수를 완전히 버리지 않고, 그 변수가 남긴 정보를 prior factor로 압축하는 과정이다. Sliding window는 계산량을 제한하기 위해 필요하지만, 과거 정보를 모두 버리면 drift가 커진다.

간단 예시: 요약 노트 비유

과거 frame 0을 window에서 빼야 한다고 하자. 그 frame을 그냥 삭제하면 frame 0이 알려준 geometry 정보를 잃는다. Marginalization은 frame 0과 연결된 정보를 “요약 노트”처럼 남겨서 다음 optimization이 그 정보를 계속 참고하게 만든다.

수식

남길 변수 a , 제거할 변수 b 로 normal equation을 나누면 다음과 같다.

$$\begin{bmatrix} H_{aa} & H_{ab} \\ H_{ba} & H_{bb} \end{bmatrix} \begin{bmatrix} \delta x_a \\ \delta x_b \end{bmatrix} = \begin{bmatrix} b_a \\ b_b \end{bmatrix}$$

제거할 변수 b 를 Schur complement로 없애면 prior가 된다.

$$H'_{aa} = H_{aa} - H_{ab}H_{bb}^{-1}H_{ba}, \quad b'_a = b_a - H_{ab}H_{bb}^{-1}b_b$$

각 항의 의미

항	의미
δx_a	window에 남길 변수 update. 최근 pose, extrinsic 등.
δx_b	제거할 변수 update. 오래된 pose, feature depth 등.
H_{aa}, H_{bb}	남길 변수/제거할 변수의 Hessian block.
H_{ab}, H_{ba}	남길 변수와 제거할 변수 사이 coupling.
H'_{aa}, b'_a	제거 후 남는 prior factor.

Schur complement 그림

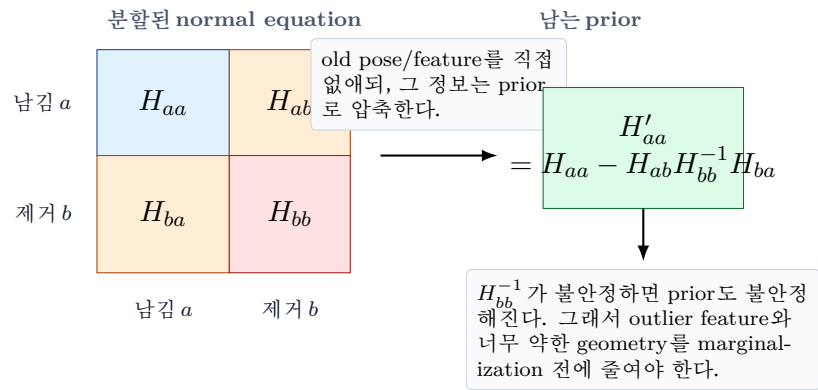


그림 8. Sliding window가 작아도 과거 정보를 완전히 버리지 않기 위해 Schur complement로 prior factor를 만든다.

파라미터와 영향

항목	수식항 관계	작게/부족하면	크게/많으면
WINDOW_SIZE	H matrix 크기	빠르지만 drift 억제 약함.	constraint 증가, 계산량/memory 증가.
MARGIN_OLD	오래된 x_b 제거	과거 정보가 빨리 prior 화됨.	선형화 prior 가 누적될 수 있음.
MARGIN_SECOND_NEW	새 frame 근처 제거	keyframe 보존 부족 시 window 품질 저하.	최신 motion 반영이 둔해질 수 있음.
keyframe_parallax	marginalization 방향에 간접 영향	keyframe 과다, 계산량 증가.	keyframe 부족, geometry 악화.

실제 코드

읽는 포인트: 제거할 변수와 연결된 residual을 Schur complement로 압축해 다음 window의 prior로 넘긴다.

estimator.cpp:1140-1255

```

1140     if (marginalization_flag == MARGIN_OLD)
1141     {
1142         MarginalizationInfo *marginalization_info = new MarginalizationInfo();
1143         vector2double();
1144
1145         if (last_marginalization_info && last_marginalization_info->valid)
1146         {
1147             vector<int> drop_set;
1148             for (int i = 0; i < static_cast<int>(last_marginalization_parameter_blocks.size()); i++)
1149             {
1150                 if (last_marginalization_parameter_blocks[i] == para_Pose[0] ||
1151                     last_marginalization_parameter_blocks[i] == para_SpeedBias[0])
1152                     drop_set.push_back(i);
1153             }
1154             // construct new marginalization_factor
1155             MarginalizationFactor *marginalization_factor = new MarginalizationFactor(
1156                 last_marginalization_info);
1157             ResidualBlockInfo *residual_block_info = new ResidualBlockInfo(marginalization_factor, NULL,
1158
1159                 last_marginalization_parameter_blocks,
1160
1161                                     drop_set);
1162             marginalization_info->addResidualBlockInfo(residual_block_info);
1163         }
1164
1165         if (USE_IMU)
1166         {
1167             if (pre_integrations[1]->sum_dt < 10.0)
1168             {
1169                 IMUFactor* imu_factor = new IMUFactor(pre_integrations[1]);
1170                 ResidualBlockInfo *residual_block_info = new ResidualBlockInfo(imu_factor, NULL,
1171
1172                                     vector<double *>{para_Pose[0],
1173
1174                                     para_SpeedBias[0], para_Pose[1], para_SpeedBias[1]},
1175
1176                                     vector<int>{0, 1});
1177                 marginalization_info->addResidualBlockInfo(residual_block_info);
1178             }
1179         }
1180
1181         {
1182             int feature_index = -1;
1183             for (auto &it_per_id : f_manager.feature)
1184             {
1185                 it_per_id.used_num = it_per_id.feature_per_frame.size();
1186                 if (it_per_id.used_num < 4)
1187                     continue;
1188             }
1189         }
1190     }
1191 }

```

```

1182         ++feature_index;
1183
1184         int imu_i = it_per_id.start_frame, imu_j = imu_i - 1;
1185         if (imu_i != 0)
1186             continue;
1187
1188         Vector3d pts_i = it_per_id.feature_per_frame[0].point;
1189
1190         for (auto &it_per_frame : it_per_id.feature_per_frame)
1191         {
1192             imu_j++;
1193             if(imu_i != imu_j)
1194             {
1195                 Vector3d pts_j = it_per_frame.point;
1196                 ProjectionTwoFrameOneCamFactor *f_td = new ProjectionTwoFrameOneCamFactor(pts_i,
1197                 pts_j, it_per_id.feature_per_frame[0].velocity, it_per_frame.velocity,
1198                 it_per_id.feature_per_frame[0].
1199                 cur_td, it_per_frame.cur_td);
1200                 ResidualBlockInfo *residual_block_info = new ResidualBlockInfo(f_td,
1201                 loss_function,
1202                 vector<double*>{
1203                 para_Pose[imu_i], para_Pose[imu_j], para_Ex_Pose[0], para_Feature[feature_index], para_Td[0]},
1204                 vector<int>{0,
1205                 3});
1206                 marginalization_info->addResidualBlockInfo(residual_block_info);
1207             }
1208             if(STEREO && it_per_frame.is_stereo)
1209             {
1210                 Vector3d pts_j_right = it_per_frame.pointRight;
1211                 if(imu_i != imu_j)
1212                 {
1213                     ProjectionTwoFrameTwoCamFactor *f = new ProjectionTwoFrameTwoCamFactor(pts_i,
1214                     pts_j_right, it_per_id.feature_per_frame[0].velocity, it_per_frame.velocityRight,
1215                     it_per_id.feature_per_frame[0].
1216                     cur_td, it_per_frame.cur_td);
1217                     ResidualBlockInfo *residual_block_info = new ResidualBlockInfo(f,
1218                     loss_function,
1219                     vector<double*>{para_Pose[imu_i], para_Pose[imu_j], para_Ex_Pose[0], para_Ex_Pose[1], para_Feature[feature_index
1220                     ], para_Td[0]},
1221                     vector<int>{0,
1222                     4});
1223                     marginalization_info->addResidualBlockInfo(residual_block_info);
1224                 }
1225                 else
1226                 {
1227                     ProjectionOneFrameTwoCamFactor *f = new ProjectionOneFrameTwoCamFactor(pts_i,
1228                     pts_j_right, it_per_id.feature_per_frame[0].velocity, it_per_frame.velocityRight,
1229                     it_per_id.feature_per_frame[0].
1230                     cur_td, it_per_frame.cur_td);
1231                     ResidualBlockInfo *residual_block_info = new ResidualBlockInfo(f,
1232                     loss_function,
1233                     vector<double*>{para_Ex_Pose[0], para_Ex_Pose[1], para_Feature[feature_index], para_Td[0]},
1234                     vector<int>
1235                     >{2});
1236                     marginalization_info->addResidualBlockInfo(residual_block_info);
1237                 }
1238             }
1239         }
1240
1241         TicToc t_pre_margin;
1242         marginalization_info->preMarginalize();
1243         ROS_DEBUG("pre marginalization %f ms", t_pre_margin.toc());
1244
1245         TicToc t_margin;
1246         marginalization_info->marginalize();
1247         ROS_DEBUG("marginalization %f ms", t_margin.toc());
1248
1249         std::unordered_map<long, double*> addr_shift;
1250         for (int i = 1; i <= WINDOW_SIZE; i++)
1251         {
1252             addr_shift[reinterpret_cast<long>(para_Pose[i])] = para_Pose[i - 1];
1253         }

```



```

1241         if(USE_IMU)
1242             addr_shift[reinterpret_cast<long>(para_SpeedBias[i])] = para_SpeedBias[i - 1];
1243     }
1244     for (int i = 0; i < NUM_OF_CAM; i++)
1245         addr_shift[reinterpret_cast<long>(para_Ex_Pose[i])] = para_Ex_Pose[i];
1246
1247     addr_shift[reinterpret_cast<long>(para_Td[0])] = para_Td[0];
1248
1249     vector<double *> parameter_blocks = marginalization_info->getParameterBlocks(addr_shift);
1250
1251     if (last_marginalization_info)
1252         delete last_marginalization_info;
1253     last_marginalization_info = marginalization_info;
1254     last_marginalization_parameter_blocks = parameter_blocks;

```

코드 분석

- sliding window 밖으로 나갈 pose/feature를 그냥 버리지 않고, 그 정보량을 prior factor로 압축한다.
- 코드 흐름은 residual block 수집 → marginalization parameter block 지정 → Schur complement 수행이다.
- marginalization은 선형화된 prior를 남기므로, outlier가 들어간 상태에서 marginalize하면 잘못된 정보도 prior로 굳어진다. 수중 repeated pattern outlier가 특히 위험한 이유다.

estimator.cpp:1329-1368

```

1329 void Estimator::slideWindow()
1330 {
1331     TicToc t_margin;
1332     if (marginalization_flag == MARGIN_OLD)
1333     {
1334         double t_0 = Headers[0];
1335         back_R0 = Rs[0];
1336         back_P0 = Ps[0];
1337         if (frame_count == WINDOW_SIZE)
1338         {
1339             for (int i = 0; i < WINDOW_SIZE; i++)
1340             {
1341                 Headers[i] = Headers[i + 1];
1342                 Rs[i].swap(Rs[i + 1]);
1343                 Ps[i].swap(Ps[i + 1]);
1344                 if(USE_IMU)
1345                 {
1346                     std::swap(pre_integrations[i], pre_integrations[i + 1]);
1347
1348                     dt_buf[i].swap(dt_buf[i + 1]);
1349                     linear_acceleration_buf[i].swap(linear_acceleration_buf[i + 1]);
1350                     angular_velocity_buf[i].swap(angular_velocity_buf[i + 1]);
1351
1352                     Vs[i].swap(Vs[i + 1]);
1353                     Bas[i].swap(Bas[i + 1]);
1354                     Bgs[i].swap(Bgs[i + 1]);
1355                 }
1356             }
1357             Headers[WINDOW_SIZE] = Headers[WINDOW_SIZE - 1];
1358             Ps[WINDOW_SIZE] = Ps[WINDOW_SIZE - 1];
1359             Rs[WINDOW_SIZE] = Rs[WINDOW_SIZE - 1];
1360
1361             if(USE_IMU)
1362             {
1363                 Vs[WINDOW_SIZE] = Vs[WINDOW_SIZE - 1];
1364                 Bas[WINDOW_SIZE] = Bas[WINDOW_SIZE - 1];
1365                 Bgs[WINDOW_SIZE] = Bgs[WINDOW_SIZE - 1];
1366
1367                 delete pre_integrations[WINDOW_SIZE];
1368                 pre_integrations[WINDOW_SIZE] = new IntegrationBase{acc_0, gyr_0, Bas[WINDOW_SIZE], Bgs[
WINDOW_SIZE]};

```

코드 분석

- window state를 한 칸씩 shift 한다. pose, velocity, bias, preintegration buffer가 함께 이동해야 시간 순서가 유지된다.
- 이 과정이 잘못되면 다음 frame의 residual이 이전 state와 맞지 않는다. 그래서 marginalization flag와 window shift는 한 세트로 이해해야 한다.
- keyframe 선택이 불안정하면 window 안에 남는 frame 구성이 나빠지고, 이후 optimization의 geometry가 약해진다.

14 9단계: Loop/global/sonar constraint

핵심 정의: Global constraint

VIO는 기본적으로 local odometry라서 시간이 지나면 drift가 누적된다. Global constraint는 loop closure, sonar landmark, depth sensor, marker처럼 절대적이거나 장기적인 기준을 제공하는 제약이다. 이 제약이 있어야 VIO가 SLAM에 가까워진다.

간단 예시: vision은 local, sonar는 global

수영장 타일은 반복되어 vision만으로는 장거리 위치를 안정적으로 고정하기 어렵다. 반면 sonar가 벽, 부표, 구조물과의 거리를 안정적으로 준다면, 이 measurement를 global factor로 넣어 local VIO drift를 잡을 수 있다. 단, sonar association이 틀리면 global factor가 오히려 path를 잘못 끌고 간다.

수식

VIO는 local odometry이므로 장시간 drift한다. SLAM에 가까워지려면 loop closure나 sonar/global measurement가 residual로 추가되어야 한다.

$$r_{\text{global}} = h(\mathbf{x}_k, \mathbf{m}) - \mathbf{z}_k$$

depth sensor 예시는 다음과 같다.

$$r_{\text{depth}} = \mathbf{e}_z^T \mathbf{p}_k - z_k^{\text{meas}}$$

각 항과 파라미터

항/파라미터	의미	영향
$h(\mathbf{x}_k, \mathbf{m})$	상태와 map으로 예측한 sonar/global measurement.	sonar model이 잘못되면 global residual이 pose를 잘못 끌고 간다.
$\mathbf{z}_k$	실제 sonar/depth/acoustic/marker measurement.	outlier handling이 필요하다.
measurement noise σ	residual weight.	작으면 sensor 과신, 크면 drift 보정 약화.
association threshold	map landmark와 measurement 연결 허용 범위.	작으면 정상 관측 reject, 크면 잘못된 landmark 연결.
robust loss scale	global outlier 억제 강도.	작으면 정상 correction도 약화, 크면 outlier에 취약.

실제 코드

읽는 포인트: VIO local path를 global pose graph로 보정하는 API 경계다. sonar/global factor도 같은 관점에서 추가한다.

pose_graph.h:46-75

```

46 class PoseGraph
47 {
48 public:
49     PoseGraph();
50     ~PoseGraph();
51     void registerPub(ros::NodeHandle &n);
52     void addKeyFrame(KeyFrame* cur_kf, bool flag_detect_loop);
53     void loadKeyFrame(KeyFrame* cur_kf, bool flag_detect_loop);
54     void loadVocabulary(std::string voc_path);
55     void setIMUFlag(bool _use_imu);
56     KeyFrame* getKeyFrame(int index);

```

```

57  nav_msgs::Path path[10];
58  nav_msgs::Path base_path;
59  CameraPoseVisualization* posegraph_visualization;
60  void savePoseGraph();
61  void loadPoseGraph();
62  void publish();
63  Vector3d t_drift;
64  double yaw_drift;
65  Matrix3d r_drift;
66  // world frame( base sequence or first sequence)<----> cur sequence frame
67  Vector3d w_t_vio;
68  Matrix3d w_r_vio;
69
70
71 private:
72  int detectLoop(KeyFrame* keyframe, int frame_index);
73  void addKeyFrameIntoVoc(KeyFrame* keyframe);
74  void optimize4DoF();
75  void optimize6DoF();

```

코드 분석

- PoseGraph는 local VIO 결과를 keyframe 단위 graph로 저장하고, loop constraint를 추가해 global drift를 줄이는 구조다.
- VINS estimator가 frame-to-frame local odometry를 만들고, pose graph는 더 긴 시간 범위의 일관성을 본다. 이 둘을 구분해야 한다.
- sonar/global landmark를 넣는다면 새 sensor residual도 결국 pose graph 또는 backend factor 형태로 들어가야 한다. 단순히 RViz path를 후처리로 보정하는 것과 다르다.

keyframe.cpp:483-501

```

483  if ((int)matched_2d_cur.size() > MIN_LOOP_NUM)
484  {
485      relative_t = PnP_R_old.transpose() * (origin_vio_T - PnP_T_old);
486      relative_q = PnP_R_old.transpose() * origin_vio_R;
487      relative_yaw = Utility::normalizeAngle(Utility::R2ypr(origin_vio_R).x() - Utility::R2ypr(PnP_R_old).x());
488      //printf("PNP relative\n");
489      //cout << "pnp relative_t " << relative_t.transpose() << endl;
490      //cout << "pnp relative_yaw " << relative_yaw << endl;
491      if (abs(relative_yaw) < 30.0 && relative_t.norm() < 20.0)
492      {
493
494          has_loop = true;
495          loop_index = old_kf->index;
496          loop_info << relative_t.x(), relative_t.y(), relative_t.z(),
497                      relative_q.w(), relative_q.x(), relative_q.y(), relative_q.z(),
498                      relative_yaw;
499          //cout << "pnp relative_t " << relative_t.transpose() << endl;
500          //cout << "pnp relative_q " << relative_q.w() << " " << relative_q.vec().transpose() << endl;
501          return true;

```

코드 분석

- loop가 검출되면 두 keyframe 사이의 상대 pose constraint를 만든다. 이 constraint가 pose graph optimization에서 drift를 당겨준다.
- 수영장처럼 반복 패턴이 많은 곳에서는 loop detection 자체도 false positive가 날 수 있다. global constraint에는 robust loss와 추가 검증이 필요하다.
- sonar나 marker를 쓴다면 loop image matching보다 더 신뢰도 높은 global edge로 쓸 수 있다. 다만 measurement noise를 작게 잡으면 잘못된 sonar association이 전체 path를 망칠 수 있다.

15 10단계: 현재 GUI wrapper의 위치

GUI는 VINS 알고리즘의 핵심이 아니라 실험 자동화 wrapper다. 다음 일을 수행한다.

1. 좌/우 MP4를 KITTI-style image sequence로 변환한다.
2. camera/tracker/backend 파라미터를 VINS YAML로 만든다.

3. Docker 안에서 VINS-Fusion kitti_odom_test를 실행한다.
4. vio.txt를 CSV로 바꿔 RViz live path로 넘긴다.

GUI 값	VINS 연결	주의점
frame/fps	sequence timestamp	IMU 결합 시 timestamp 정확도가 중요하다.
camera calib	stereo projection/depth	수중 calibration 없이 metric trajectory를 믿으면 안 된다.
tracker params	feature tracking	반복 타일에서는 feature 수보다 품질과 분산이 중요하다.
solver params	backend runtime	대회용 realtime성과 수렴 품질의 trade-off다.

읽는 포인트: GUI는 알고리즘 자체가 아니라 dataset/config/run wrapper다.

web_gui.py:658-736

```

658 658 은기준입니다여기서는를고르지않고의파라미터만조정합니다정확한이없으면는참고용입니다를모르면로임시생성합니
다이크져있어야합니다전체프레임은시간이걸리므로먼저프레임으로확인하세요를선택하면원본영상이표시됩니다
를누르면결과가생성됩니다이화면은실행중입니다없이만쓰기때문에실제대회용까지가려면카메라와연결이다른단계
입니다결과를로확인합니다다가켜져있으면방금로바로갱신됩니다만생성했습니다를누르면에서실행합니다
659 def make_vins_config(payload: dict[str, Any], run_vins: bool) -> dict[str, Any]:
660     left_video = _require_project_file(payload["left_video"])
661     right_video = _require_project_file(payload["right_video"])
662     if left_video == right_video:
663         raise ValueError("VINS stereo needs different left/right MP4 files.")
664
665     timestamp = time.strftime("run_%Y%m%d_%H%M%S") + f"_{int(time.time() * 1000) % 1000:03d}"
666     output_root = PROJECT_ROOT / "outputs" / "vins_fusion" / "gui_runs" / timestamp
667     dataset_dir = output_root / "underwater_stereo_kitti"
668     config_dir = output_root / "config"
669     output_dir = output_root / "vins_output"
670     odometry_csv = output_root / "vins_odometry.csv"
671
672     export_cmd = [
673         sys.executable,
674         str(PROJECT_ROOT / "scripts" / "export_vins_kitti_stereo.py"),
675         "--left-video",
676         str(left_video),
677         "--right-video",
678         str(right_video),
679         "--output-dir",
680         str(dataset_dir),
681         "--max-frames",
682         str(int(payload.get("max_frames"))),
683         "--start-frame",
684         str(int(payload.get("start_frame", 0))),
685         "--stride",
686         str(int(payload.get("stride", 1))),
687         "--resize-width",
688         str(int(payload.get("resize_width"))),
689         "--force",
690     ]
691     fps = _optional_float(payload.get("fps"))
692     if fps and fps > 0:
693         export_cmd.extend(["--fps", str(fps)])
694     export_stdout = _run_command(export_cmd, timeout=180)
695     metadata = json.loads(export_stdout)
696
697     fx = _optional_float(payload.get("fx"))
698     fy = _optional_float(payload.get("fy"))
699     cx = _optional_float(payload.get("cx"))
700     cy = _optional_float(payload.get("cy"))
701     config_cmd = [
702         sys.executable,
703         str(PROJECT_ROOT / "scripts" / "vins_make_stereo_config.py"),
704         "--metadata",
705         str(dataset_dir / "metadata.json"),
706         "--config-dir",
707         str(config_dir),
708         "--output-dir",
709         str(output_dir),
710         "--focal-scale",

```

```

711     str(float(payload["focal_scale"])),
712     "--baseline",
713     str(float(payload["baseline"])),
714     "--max-cnt",
715     str(int(payload["max_cnt"])),
716     "--min-dist",
717     str(int(payload["min_dist"])),
718     "--freq",
719     str(int(payload.get("freq", 10))),
720     "--f-threshold",
721     str(float(payload["f_threshold"])),
722     "--max-solver-time",
723     str(float(payload["max_solver_time"])),
724     "--max-num-iterations",
725     str(int(payload["max_num_iterations"])),
726     "--keyframe-parallax",
727     str(float(payload["keyframe_parallax"])),
728 ]
729 if fx and fx > 0:
730     config_cmd.extend(["--fx", str(fx)])
731 if fy and fy > 0:
732     config_cmd.extend(["--fy", str(fy)])
733 if cx is not None:
734     config_cmd.extend(["--cx", str(cx)])
735 if cy is not None:
736     config_cmd.extend(["--cy", str(cy)])
737 config_file = Path(_run_command(config_cmd, timeout=60).strip()).resolve()

```

코드 분석

- GUI에서 받은 값을 dataset export와 VINS config 생성으로 연결하는 wrapper다. VINS 알고리즘을 새로 구현하는 부분은 아니다.
- 여기서 만든 YAML 값이 parameters.cpp의 readParameters()로 들어가므로, GUI 조작은 결국 VINS 전역 파라미터 변경으로 이어진다.
- 수중 실험에서는 GUI에 보이는 값과 실제 생성된 YAML을 항상 같이 확인해야 한다. 특히 fx/fy/cx/cy, baseline, resize는 depth scale에 직접 연결된다.

web_gui.py:750-775

은기존입니다여기서는를고르지않고의파라미터만조정합니다정확한이없으면는참고용입니다를모르면로임시생성합니다
다익쳐져있어야합니다전체프레임은시간이걸리므로먼저프레임으로확인하세요를선택하면원본영상이표시됩니다
를누르면영상가생성됩니다다익화면은실행중입니다없이만쓰기때문에실제대회용까지가려면카메라과연결이다음단계
입니다결과를로확인합니다가켜져있으면방금로바로갱신됩니다만생성했습니다를누르면에서을실행합니다

```

750 docker_cmd = [
751     str(PROJECT_ROOT / "scripts" / "vins_docker_run_stereo_kitti.sh"),
752     str(dataset_dir),
753     str(config_file),
754 ]
755 result["vins_log"] = _tail_lines(_run_command(docker_cmd, timeout=900), 80)
756
757 convert_cmd = [
758     sys.executable,
759     str(PROJECT_ROOT / "scripts" / "convert_vins_vio_to_csv.py"),
760     "--vio-txt",
761     str(output_dir / "vio.txt"),
762     "--times-txt",
763     str(dataset_dir / "times.txt"),
764     "--output-csv",
765     str(odometry_csv),
766 ]
767 _run_command(convert_cmd, timeout=60)
768 result["odometry_csv"] = str(odometry_csv)
769 result["pose_count"] = max(0, sum(1 for _ in odometry_csv.open(encoding="utf-8")) - 1)
770
771 live_dir = PROJECT_ROOT / "outputs" / "live"
772 live_dir.mkdir(parents=True, exist_ok=True)
773 live_odometry = live_dir / "latest_odometry.csv"
774 _atomic_copy(odometry_csv, live_odometry)
775 result["live_odometry_csv"] = str(live_odometry)

```

코드 분석

- VINS 실행 결과를 CSV/RViz live file로 넘기는 후처리 구간이다. 즉 RViz에서 보는 path는 VINS가 낸 `vio.txt`를 변환한 결과다.
- Docker/VINS 실행이 실패하면 이 단계에는 실제 odometry가 없고, RViz도 갱신할 path가 없다. 이전에 보던 live path가 남아 있으면 결과를 착각할 수 있다.
- GUI에서 “Run selected”를 눌렀을 때 봐야 하는 것은 영상 preview, 생성 config, VINS stdout, `vio.txt` 존재 여부, RViz path 갱신 순서다.

16 수중 UUV 개발 순서

1. 수중 stereo calibration을 먼저 확보한다. K , distortion, baseline이 틀리면 모든 depth와 scale이 틀어진다.
2. stereo-only VINS를 짧은 구간에서 검증한다. feature track, depth, scale, RViz path 방향을 본다.
3. IMU topic을 연결한다. imu: 1, camera-IMU extrinsic, time offset, noise를 맞춘다.
4. feature frontend를 수영장 환경에 맞춘다. MIN_DIST, MAX_CNT, FLOW_BACK을 조절한다.
5. backend realtime성을 맞춘다. solver_time, iteration, frame rate를 대회용 속도에 맞춘다.
6. sonar/depth/global factor를 추가해 local drift를 줄인다.

17 핵심 요약

VINS-Fusion을 제대로 읽는 핵심은 다음 관계를 계속 따라가는 것이다.

parameter → equation term → residual → trajectory behavior

수중 UUV에서는 특히 다음 세 가지가 중요하다.

1. feature는 많이 잡는 것보다 오래 살아남고 넓게 퍼지는 것이 중요하다.
2. stereo depth는 $Z = f_x b / d$ 라서 calibration과 disparity error에 매우 민감하다.
3. VIO-SLAM은 visual residual만으로 끝나지 않고 IMU와 sonar/global residual을 factor graph에 함께 넣어야 한다.